	THE UNIVERSITY OF SHEFFIELD **School of Electrical and Electronic Engineering** **Individual Project** **Dissertation**		
Student Name	Yuqi Guo (ID: 230184273)		
Project Title	Hardware Acceleration for Cluster Fault Tolerance in Hybrid CMOS non CMOS Memories		
Supervisor	Mr. Neil Powell	**Second Marker**	Dr. Mohammad Eissa

Hardware Acceleration for Cluster Fault Tolerance in Hybrid CMOS non CMOS Memories

Abstract

Hybrid CMOS/non-CMOS memories offer the prospect of ultra-high-density data storage, but are inherently susceptible to cluster faults — spatially correlated bursts of multiple consecutive bit errors — arising from the dense packing of nanoscale devices. Redundant Residue Number System (RRNS) codes have been identified as particularly well-suited for cluster fault tolerance, as each residue occupies a contiguous bit field and a burst error typically corrupts only a small number of residues regardless of burst length. However, the conventional RRNS (C-RRNS) implementation incurs a 61-bit codeword for 16-bit data — 27.1% longer than the equivalent Reed-Solomon (RS) code — motivating the development of two modified variants: Three Non-Redundant Moduli RRNS (3NRM-RRNS) and Two Non-Redundant Moduli RRNS (2NRM-RRNS).

This dissertation presents the design and implementation of an FPGA-based hardware acceleration platform for evaluating the fault tolerance performance of seven ECC algorithm configurations — including both parallel and serial implementations of 2NRM-RRNS — on a Xilinx Artix-7 device. A novel probabilistic fault injection engine is developed, enabling statistically rigorous BER testing (0–10%, 101 points, 100,000 samples per point) under both random single-bit and cluster burst fault models using only two Block RAMs. A unified encoder/decoder wrapper architecture provides compile-time algorithm selection, ensuring fair resource comparison through single-algorithm builds.

Key results demonstrate that C-RRNS-MLD achieves 100% decode success across the full 0–10% BER range under random single-bit injection — a finding consistent with its t=3 correction capability and confirmed by hardware measurement. The 2NRM-RRNS parallel decoder achieves the lowest latency among correcting algorithms (24 clock cycles at 50 MHz) while

providing the best storage efficiency (39.0%, 41-bit codeword). A direct comparison of parallel and serial 2NRM-RRNS implementations quantifies the resource-latency trade-off: the parallel architecture achieves 15× lower latency at the cost of approximately 8–10× higher LUT utilisation, with identical BER performance confirming implementation correctness.

Individual Contribution

This project was undertaken independently as an individual final-year dissertation. All work described in this report was performed solely by the author, Yuqi Guo (Student ID: 230184273), under the supervision of Mr. Neil Powell at the University of Sheffield.

MATLAB Simulation Phase: The MATLAB simulation framework (Section 3.1) was designed and implemented from scratch by the author. This includes the Monte Carlo test harness, the encoding and decoding implementations for all four algorithms (RS, C-RRNS, 3NRM-RRNS, 2NRM-RRNS), the fault injection model, and the performance analysis scripts. The MATLAB built-in Reed-Solomon codec functions were used for the RS algorithm baseline, as acknowledged in the Interim Report.

FPGA Implementation Phase: All Verilog/SystemVerilog source code described in Chapter 3 was written by the author. This includes:

- Encoder modules for all four algorithm families (`encoder_2nrm` , `encoder_3nrm` , `encoder_crrns` , `encoder_rs`)
- Decoder modules for all seven configurations, including the novel parallel MLD pipeline (`decoder_2nrm`) and serial FSM (`decoder_2nrm_serial`)
- The probabilistic fault injection engine (`error_injector_unit` , `rom_threshold_ctrl`)
- The test infrastructure (`auto_scan_engine` , `main_scan_fsm` , `seed_lock_unit` , `mem_stats_array` , `tx_packet_assembler`)
- The UART communication layer (`uart_rx_module` , `uart_tx_module` , `protocol_parser` , `ctrl_register_bank`)

PC-Side Software: All Python scripts for test control, data collection, and visualisation (`py_controller_main.py` , `gen_rom.py` , `compare_ber_curves.py` , `plot_utilization.py` , `plot_latency.py` , `plot_storage_efficiency.py`) were written by the author.

Hardware Resources: The Xilinx Artix-7 xc7a100t FPGA (Arty A7-100T development board) was used as the target platform. The Xilinx Vivado 2023.2 design suite was used for synthesis, implementation, and bitstream generation under a standard academic licence. No external datasets were used; all experimental data was collected by the author using the platform described in this report.

The algorithms evaluated in this work are based on the published research of Haron and Hamdioui [1] and Goh and Siddiqi [2], as cited throughout the dissertation.

Acknowledgement

I would like to express my sincere gratitude to my project supervisor, Mr. Neil Powell, for his invaluable guidance, encouragement, and constructive feedback throughout the duration of this project. His expertise in digital systems design and his patient support during the FPGA implementation phase were instrumental in shaping both the technical direction and the quality of this work. I am particularly grateful for his willingness to engage with the detailed hardware debugging challenges encountered during the development of the fault injection platform, and for his insightful comments on the interim report that helped focus the scope of the final dissertation.

I would also like to thank Dr. Mohammad Eissa, the second marker for this project, for his time and effort in reviewing this dissertation.

Finally, I would like to thank my family for their unwavering support and encouragement throughout my studies at the University of Sheffield.

Chapter 1 Introduction

1.1 Background

The relentless scaling of CMOS transistor geometry and the emergence of non-CMOS nanodevices have opened the prospect of hybrid CMOS/non-CMOS memories capable of storing data at densities approaching 1 Tbit/cm² [1]. In these hybrid architectures, arrays of nanowire crossbars are fabricated on top of nanoscale CMOS circuits. At each crosspoint, a two-terminal nanodevice — such as a single-electron junction, organic molecule, or phase-change material — serves as a single-bit memory cell. The CMOS layer performs peripheral functions (encoding, decoding, sensing, and global interconnection), while the nanoscale crossbar provides ultra-high-density storage.

Despite their extraordinary capacity potential, hybrid memories are inherently susceptible to two categories of faults. First, **manufacturing defects** arise from the imprecision of top-down fabrication techniques (e.g., extreme ultraviolet lithography, nanoimprint) and the immaturity of bottom-up self-assembly processes, leading to broken nanowires, missing nanodevices, and misaligned interface pins. Second, **transient faults** occur during operation due to the reduced signal-to-noise ratio caused by lower supply voltages and smaller capacitances at nanoscale

dimensions. Charged-based non-CMOS nanodevices are particularly vulnerable because they require only a small voltage perturbation to change their internal state.

A critical characteristic of these faults is their **spatial correlation**: because nanodevices are densely packed and closely interconnected, a single fault event can propagate to affect several contiguous memory cells, resulting in **cluster faults** — spatially correlated bursts of multiple consecutive bit errors. In the literature, the terms *cluster faults*, *cluster errors*, and *burst errors* are used interchangeably to describe this phenomenon; this dissertation adopts the term **cluster faults** (following the terminology of Haron and Hamdioui [1]) when referring to the fault model, and **burst errors** or **burst injection** when referring specifically to the fault injection mechanism used in the evaluation platform. This fault model is fundamentally different from the independent random bit errors assumed by conventional error-correcting codes (ECCs) such as Hamming codes, BCH codes, and Euclidean Geometry codes, which have been widely applied to hybrid memories [6–11] but were designed for random, uncorrelated errors.

The Redundant Residue Number System (RRNS) was identified by Haron and Hamdioui [1] as a particularly suitable ECC for cluster fault tolerance in hybrid memories. Unlike conventional ECCs, RRNS operates on residue representations of data, where each residue is computed independently with respect to a different modulus. A cluster fault that corrupts a contiguous block of bits will typically corrupt only a small number of residues (since each residue occupies a contiguous bit field), regardless of the burst length. This property makes RRNS inherently well-suited for cluster fault correction.

However, the conventional RRNS (C-RRNS) implementation incurs significant storage overhead: for 16-bit data, C-RRNS requires a 61-bit codeword — 27.1% longer than the 48-bit Reed-Solomon (RS) codeword for the same data width. This overhead is a direct consequence of the requirement that redundant moduli must be larger than non-redundant moduli, which forces the use of large integers and long residue fields. Reducing this overhead while maintaining competitive error correction capability is identified as an open challenge in the literature [8].

To address this limitation, Haron and Hamdioui [1] proposed two modified RRNS variants — **Three Non-Redundant Moduli RRNS (3NRM-RRNS)** and **Two Non-Redundant Moduli RRNS (2NRM-RRNS)** — that use smaller redundant moduli to achieve shorter codewords. These variants require Maximum Likelihood Decoding (MLD) to resolve the decoding ambiguity introduced by the violation of the conventional moduli ordering rule, but offer significant advantages in storage efficiency and decoding speed.

While other advanced ECC schemes have been considered for hybrid memory protection, each exhibits limitations in the cluster fault context. Low-Density Parity-Check (LDPC) codes and Turbo codes offer excellent random error correction performance but incur high decoding complexity and iterative latency that is incompatible with the low-latency access requirements of

memory systems [8]. BCH codes, though hardware-efficient, are similarly optimised for random rather than spatially correlated errors. Furthermore, existing FPGA implementations of RRNS-based ECC [3] have focused on single-algorithm implementations without systematic multi-algorithm comparison, and no prior work has quantified the resource-latency trade-off between parallel and sequential MLD architectures on physical hardware. This work addresses these gaps directly.

While the theoretical properties of these algorithms have been established through MATLAB simulation [1], their practical feasibility on hardware platforms — including resource utilisation, timing characteristics, and actual BER performance under realistic fault injection — has not been comprehensively evaluated. This gap motivates the present work.

1.2 Aims and Objectives

The primary aim of this project is to design and implement an FPGA-based hardware acceleration platform for evaluating the fault tolerance performance of multiple ECC algorithms under realistic cluster fault conditions, and to use this platform to provide a comprehensive, hardware-validated comparison of the 2NRM-RRNS, 3NRM-RRNS, C-RRNS, and RS(12,4) algorithms.

The specific objectives are:

1. **Algorithm Validation**: Quantitatively evaluate the error correction capability and performance advantages of 3NRM-RRNS and 2NRM-RRNS codes through both MATLAB simulation and FPGA hardware implementation, with specific focus on cluster fault tolerance compared to C-RRNS and RS codes.
2. **Hardware Implementation**: Design and implement efficient encoder/decoder architectures for all four coding schemes (RS, C-RRNS, 3NRM-RRNS, 2NRM-RRNS) using Verilog HDL on a Xilinx Artix-7 FPGA (Arty A7-100T development board).
3. **Fault Injection Platform**: Develop a novel probabilistic fault injection engine capable of evaluating algorithm performance under both random single-bit and cluster burst fault models, with configurable burst lengths (1–15 bits) and BER sweep from 0% to 10%.
4. **Performance Benchmarking**: Conduct comprehensive performance analysis including:
   - BER vs. decode success rate under three fault injection scenarios
   - Encoder and decoder processing latency (clock cycles)
   - FPGA resource utilisation (LUT, FF, DSP, BRAM)
   - Storage efficiency (codeword length vs. data width)
5. **Architectural Exploration**: Implement the 2NRM-RRNS decoder in both parallel (15-channel MLD pipeline) and serial (sequential FSM) architectures to directly quantify the

resource-latency trade-off of parallel MLD.

1.3 Expected Contributions

The expected contributions of this work are:

- **Comprehensive hardware-validated performance characterisation** of novel RRNS variants (2NRM-RRNS, 3NRM-RRNS) under realistic cluster fault models on an FPGA platform.
- **A novel probabilistic fault injection engine** that enables statistically rigorous BER testing with arbitrary sample counts using minimal hardware resources (two Block RAMs).
- **A novel parallel vs. serial MLD comparison** for the 2NRM-RRNS algorithm, providing direct quantification of the resource-latency trade-off.
- **Open-source Verilog implementations** of optimised RRNS encoder/decoder architectures with a unified, extensible wrapper interface.
- **Benchmark data** comparing RRNS approaches with RS codes across four evaluation dimensions: fault tolerance, processing latency, resource utilisation, and storage efficiency.

1.4 Report Structure

The remainder of this dissertation is organised as follows:

- **Chapter 2** provides the theoretical background for all four ECC algorithms, including the mathematical foundations of RNS, RRNS encoding/decoding, the C-RRNS, 3NRM-RRNS, 2NRM-RRNS, and RS(12,4) algorithms, and the Maximum Likelihood Decoding method.
- **Chapter 3** describes the methodology, covering the MATLAB simulation phase (Section 3.1) and the FPGA implementation phase (Section 3.2), including the top-level system architecture, the probabilistic fault injection engine, and the end-to-end test loop.
- **Chapter 4** presents and discusses the experimental results across all four evaluation dimensions.
- **Chapter 5** analyses the economic, legal, social, ethical, and environmental context of this work.
- **Appendices** provide pseudocode descriptions of all encoder and decoder implementations.

Chapter 2 Theoretical Background

2.1 Residue Number System (RNS) Fundamentals

The Residue Number System (RNS) is a non-weighted numeral representation system based on modular arithmetic, first formalised by Garner [5]. In an RNS, an integer X is represented by a set of residues $(x_1, x_2, \ldots, x_n)$, where each residue $x_i = X \bmod m_i$ is computed with respect to a chosen modulus m_i. The moduli set $\{m_1, m_2, \ldots, m_n\}$ must satisfy three conditions:

1. **Pairwise coprimality**: $\gcd(m_i, m_j) = 1$ for all $i \neq j$.
2. **Strict ordering**: $m_1 < m_2 < \cdots < m_n$.
3. **Dynamic range sufficiency**: The product $M = \prod_{i=1}^{n} m_i$ must be sufficient to represent all numbers in the legitimate range $[0, M_a - 1]$.

By the Chinese Remainder Theorem (CRT), any integer X in the range $[0, M-1]$ is uniquely determined by its residue representation. This property enables parallel arithmetic operations — addition, subtraction, and multiplication — to be performed independently on each residue channel without carry propagation between channels, offering inherent speed advantages for digital signal processing and memory applications.

2.2 Redundant Residue Number System (RRNS) Codes

The Redundant Residue Number System (RRNS) extends the basic RNS by partitioning the moduli set into two subsets [13]:

- **Non-redundant moduli** $\{m_1, \ldots, m_k\}$: used to represent the data word (dataword).
- **Redundant moduli** $\{m_{k+1}, \ldots, m_n\}$: used to generate check residues (checkword/parity).

The error correction capability of an RRNS code is:

$$t = \left\lfloor \frac{n-k}{2} \right\rfloor$$

That is, the code can correct up to t erroneous residues, or detect up to $2t$ erroneous residues. This is equivalent to the error correction capability of Reed-Solomon codes, making RRNS a competitive alternative for cluster fault tolerance.

2.2.1 RRNS Encoding

Encoding is straightforward: for input data X, compute the residue with respect to each modulus:

$$x_i = X \bmod m_i, \quad i = 1, 2, \ldots, n$$

The resulting n-tuple $(x_1, x_2, \ldots, x_n)$ is the RRNS codeword. The non-redundant residues $(x_1, \ldots, x_k)$ represent the data, and the redundant residues $(x_{k+1}, \ldots, x_n)$ serve as parity.

2.2.2 RRNS Decoding

Decoding proceeds in two phases: error detection and error correction.

Error Detection: The received codeword is decoded to a value X_n using all n residues. If $X_n \leq M_a$ (the product of the non-redundant moduli), the codeword is valid and no correction is needed. If $X_n > M_a$, errors are detected.

Error Correction: A trial-and-error procedure is applied. For each combination of $z = n - t$ residues (discarding t residues at a time), the data is reconstructed as X_z and compared against the product of the remaining moduli M_z. If $X_z \leq M_z$, the correct data has been recovered. The maximum number of iterations is $\binom{n}{t}$.

Two reconstruction algorithms are available: the **Chinese Remainder Theorem (CRT)**, which uses large integer arithmetic, and **Mixed-Radix Conversion (MRC)**, which uses smaller integers and is computationally more efficient. MRC is defined as:

$$X_r = \sum_{s=1}^{n} v_s w_s$$

where the mixed-radix digits v_s are computed sequentially:

$$v_1 = x_1, \quad v_s = \left| \left((x_s - v_1) \cdot m_{1s}^{-1} - \cdots - v_{s-1} \right) \cdot m_{(s-1)s}^{-1} \right|_{m_s}$$

and the weight coefficients are $w_1 = 1$, $w_s = \prod_{i=1}^{s-1} m_i$.

2.3 Conventional RRNS (C-RRNS)

The Conventional RRNS (C-RRNS) code, as defined in [1], uses three restricted non-redundant moduli of the form $m_a = \{2^f, 2^f - 1, 2^f + 1\}$, where f is a positive integer. For 16-bit data words ($X \in [0, 65535]$), the smallest valid choice is $f = 6$, giving:

$$m_a = \{64, 63, 65\}, \quad M_a = 64 \times 63 \times 65 = 262{,}080 > 65{,}535$$

Six redundant moduli $m_b = \{67, 71, 73, 79, 83, 89\}$ are appended, all larger than the non-redundant moduli. The resulting codeword has $k + (n - k) = 3 + 6 = 9$ residues and error correction capability $t \leq 3$.

The total codeword length is $6 + 6 + 7 + 7 + 7 + 7 + 7 + 7 + 7 = 61$ bits. The requirement that redundant moduli must be larger than non-redundant moduli (Rule 2 of Section 2.1) is satisfied,

enabling standard MRC decoding without ambiguity.

Limitation: The use of large redundant moduli results in a 61-bit codeword — 27.1% longer than the 48-bit RS(12,4) codeword for the same data width. This storage overhead is the primary motivation for the modified RRNS variants described in Sections 2.5 and 2.6.

2.4 Reed-Solomon (RS) Codes

Reed-Solomon codes are a class of block error-correcting codes defined over finite fields (Galois fields) $\mathrm{GF}(2^m)$ [5]. An RS(n, k) code over $\mathrm{GF}(2^m)$ encodes k data symbols into n codeword symbols, each of m bits, with error correction capability $t = (n-k)/2$ symbols.

For the 16-bit data word comparison in this work, the RS(12, 4) code over $\mathrm{GF}(2^4)$ is used:

- 4 data symbols × 4 bits = 16 bits of data
- 8 parity symbols × 4 bits = 32 bits of parity
- Total codeword: 12 symbols × 4 bits = **48 bits**
- Error correction capability: $t = (12-4)/2 = 4$ symbols

Encoding uses systematic polynomial division: the data polynomial $d(x)$ is multiplied by x^{n-k} and divided by the generator polynomial $g(x) = \prod_{i=1}^{n-k}(x - \alpha^i)$, where α is a primitive element of $\mathrm{GF}(2^4)$. The remainder gives the parity symbols.

Decoding uses the Berlekamp-Massey algorithm to find the error locator polynomial $\sigma(x)$, followed by Chien search to locate error positions, and the Forney algorithm to compute error magnitudes. The total decoding complexity is $O(t^2)$ operations in $\mathrm{GF}(2^m)$.

RS codes are well-suited for cluster fault correction because a burst of b consecutive bit errors affects at most $\lceil b/m \rceil$ symbols, and the symbol-level correction capability is independent of the burst pattern within a symbol.

2.5 Three Non-Redundant Moduli RRNS (3NRM-RRNS)

The 3NRM-RRNS code was proposed by Haron and Hamdioui [1] as a modified RRNS variant that reduces codeword length while maintaining the same error correction capability as C-RRNS.

Key innovation: The redundant moduli are chosen to be *smaller* than the non-redundant moduli, in contrast to C-RRNS where redundant moduli must be larger. Specifically:

$$m_a = \{64, 63, 65\} \quad \text{(same as C-RRNS)}$$

$$m_b = \{31, 29, 23, 19, 17, 11\}$$

The redundant moduli are the minimum values satisfying Rules 1 and 3 of Section 2.1: they are pairwise coprime, and their product $M_b = 31 \times 29 \times 23 \times 19 \times 17 \times 11 = 73{,}465{,}381 > 65{,}535$.

The total codeword length is $6 + 6 + 7 + 5 + 5 + 5 + 5 + 5 + 4 = 48$ bits — equal to RS(12,4) and 21.3% shorter than C-RRNS. The error correction capability remains $t \leq 3$.

Violation of Rule 2: Since the redundant moduli are smaller than the non-redundant moduli, Rule 2 ($m_1 < m_2 < \cdots < m_n$) is violated. This means that some input data values may have multiple valid candidates during decoding (ambiguity). This ambiguity is resolved using Maximum Likelihood Decoding (MLD), described in Section 2.7.

2.6 Two Non-Redundant Moduli RRNS (2NRM-RRNS)

The 2NRM-RRNS code [1] further reduces the codeword length by using only *two* non-redundant moduli instead of three:

$$m_a = \{2^f + 1, 2^f\} = \{257, 256\} \quad (f = 8)$$

$$m_b = \{61, 59, 55, 53\}$$

The non-redundant moduli product is $M_a = 257 \times 256 = 65{,}792 > 65{,}535$, sufficient to represent all 16-bit data values. The redundant moduli product is
$M_b = 61 \times 59 \times 55 \times 53 = 10{,}491{,}085 > 65{,}535$.

The total codeword length is $9 + 8 + 6 + 6 + 6 + 6 = 41$ bits — 14.6% shorter than RS(12,4) and 32.8% shorter than C-RRNS. The error correction capability is $t \leq 2$ (two erroneous residues).

Storage efficiency: 2NRM-RRNS achieves the highest storage efficiency among all evaluated codes:

- 2NRM-RRNS: $16/41 = 39.0\%$
- 3NRM-RRNS: $16/48 = 33.3\%$
- RS(12,4): $16/48 = 33.3\%$
- C-RRNS: $16/61 = 26.2\%$

Decoding speed: With $t = 2$ and $n = 6$ moduli, the maximum number of MRC iterations is $\binom{6}{2} = 15$. In contrast, C-RRNS and 3NRM-RRNS require $\binom{9}{3} = 84$ iterations. Therefore, 2NRM-RRNS is 5.6 times faster than C-RRNS in the decoding process [1].

Like 3NRM-RRNS, 2NRM-RRNS violates Rule 2 and requires MLD to resolve decoding ambiguity.

2.7 Maximum Likelihood Decoding (MLD) for Modified RRNS

The violation of Rule 2 in 3NRM-RRNS and 2NRM-RRNS means that the CRT uniqueness guarantee no longer holds: multiple candidate values may satisfy the validity condition $X \leq M_a$ during decoding. Maximum Likelihood Decoding (MLD), as proposed by Goh and Siddiqi [2], resolves this ambiguity.

The MLD algorithm selects the most probable valid codeword based on Hamming distance:

$$X_{\text{correct}} = \arg\min_{X \in \mathcal{V}} d_H(R_{\text{received}},\ R_{\text{expected}}(X))$$

where:

- $\mathcal{V}$ is the set of all valid candidate values (those satisfying $X \leq M_a$)
- $R_{\text{received}} = (\tilde{x}_1, \ldots, \tilde{x}_n)$ is the received (possibly corrupted) residue vector
- $R_{\text{expected}}(X) = (X \bmod m_1, \ldots, X \bmod m_n)$ is the expected residue vector for candidate X
- $d_H(A, B) = \sum_{i=1}^{n} \mathbf{1}[a_i \neq b_i]$ is the Hamming distance (number of mismatching residues)

The candidate with the minimum Hamming distance to the received residues is selected as the decoded output. If multiple candidates share the minimum distance, a secondary criterion (lower index) is applied as a tie-breaking rule. In practice, such ties are statistically rare: for a tie to occur, two distinct candidate values must produce residue vectors that are equidistant from the received (corrupted) residue vector across all n residue positions simultaneously. Given the pseudo-random distribution of 16-bit data values and the algebraic structure of the moduli set, the probability of a tie is negligible compared to the probability of a unique minimum-distance candidate, and does not contribute measurably to the observed failure rate in the experimental results of Section 4.2.

Computational complexity: For 2NRM-RRNS, the MLD procedure evaluates at most $\binom{6}{2} \times 5 = 75$ candidates (15 modulus pairs × up to 5 candidates per pair due to the periodicity of the CRT solution). For 3NRM-RRNS and C-RRNS-MLD, the procedure evaluates $\binom{9}{3} = 84$ triplets. The Hamming distance computation for each candidate requires n modulo operations and n comparisons.

2.8 Comparison of ECC Schemes

Table 2.1 summarises the key parameters of all four ECC schemes evaluated in this work, based on the analysis in [1].

Table 2.1 Comparison of ECC schemes for 16-bit data word protection.

ECC Scheme	Non-redundant moduli	Redundant moduli	Codeword (bits)	Error correction	Storage efficiency	Decoding iterations
C-RRNS	{64, 63, 65}	{67, 71, 73, 79, 83, 89}	61	t=3 residues	26.2%	C(9,3)=84
3NRM-RRNS	{64, 63, 65}	{31, 29, 23, 19, 17, 11}	48	t=3 residues	33.3%	C(9,3)=84
2NRM-RRNS	{257, 256}	{61, 59, 55, 53}	41	t=2 residues	39.0%	C(6,2)=15
RS(12,4)	GF(2^4), 4 data symbols	8 parity symbols	48	t=4 symbols	33.3%	O(t^2)

The key trade-offs are:

- **C-RRNS** provides t=3 correction but at the highest storage cost (61 bits).
- **3NRM-RRNS** matches C-RRNS in correction capability (t=3) with a 21.3% shorter codeword, at the cost of requiring MLD.
- **2NRM-RRNS** achieves the best storage efficiency (39.0%) and fastest decoding (15 iterations), with t=2 correction capability.
- **RS(12,4)** provides the highest correction capability (t=4 symbols) with a 48-bit codeword, using well-established algebraic decoding.

Chapter3 Methodology

3.1 MATLAB Simulation Phase

3.1.1 Simulation Framework

Prior to the FPGA implementation, a MATLAB-based Monte Carlo simulation was conducted to validate the theoretical error correction capabilities of the four ECC algorithms and to establish baseline performance benchmarks. The simulation framework follows a standard encode–inject–decode–analyse pipeline, as illustrated in Figure 3.1 (reproduced from the Interim Report [Interim Report, Figure 1]).

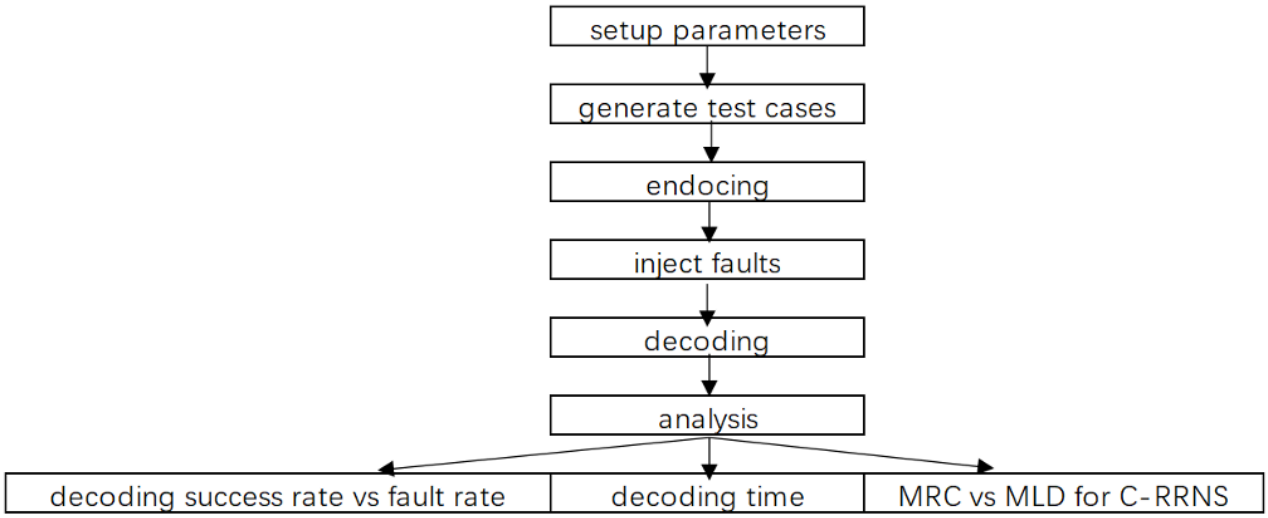


Figure 3.1 MATLAB simulation architecture. *(Reproduced from Interim Report, Figure 1.)*

The key simulation parameters are summarised in Table 3.1.

Table 3.1 MATLAB simulation parameters.

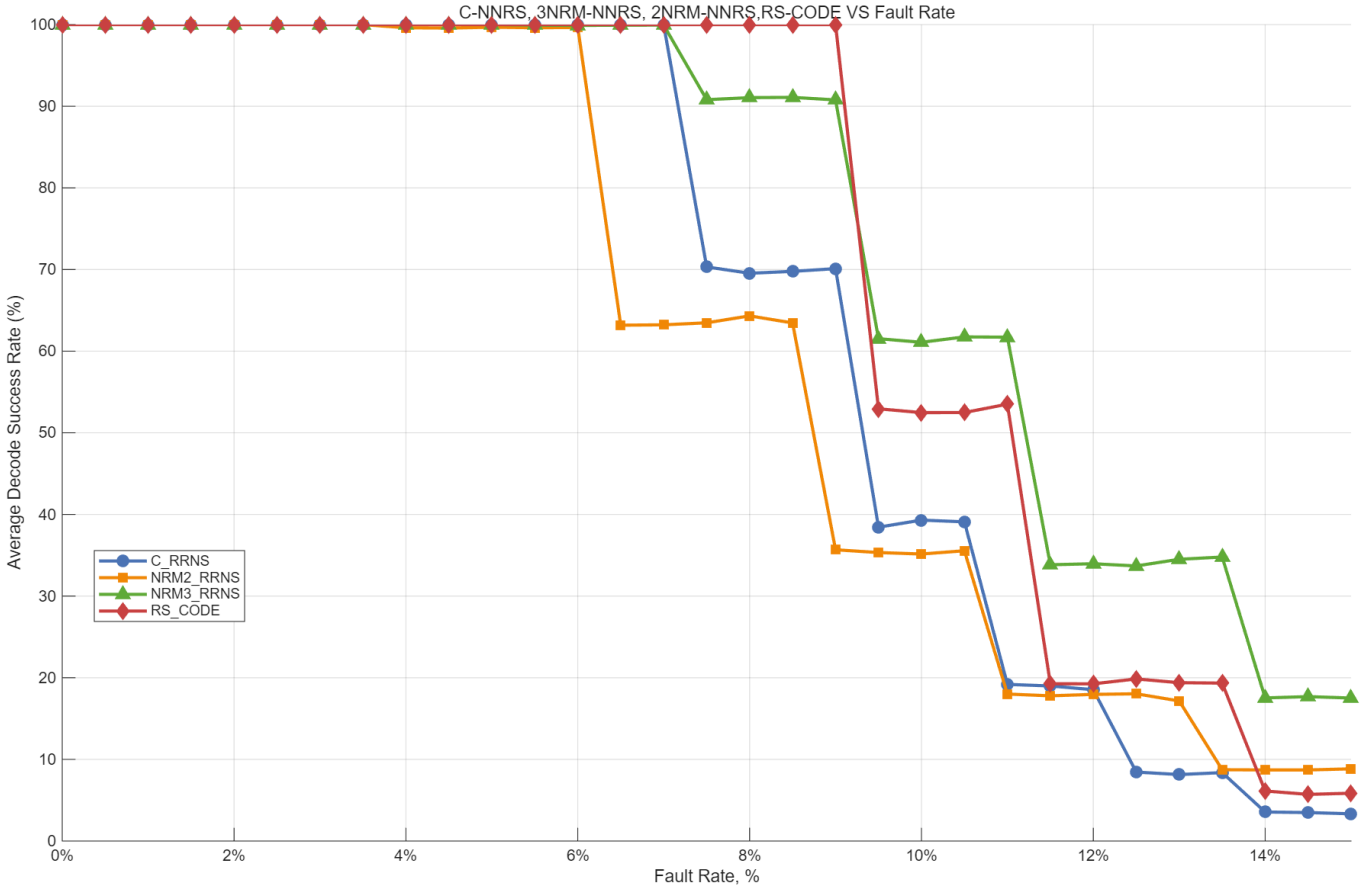
Parameter	Value
Number of Monte Carlo trials	1,000 per fault rate point
Test data words	15 random integers in [0, 65,535] per trial
Fault rate range	0 % to 15 %, step 0.5 %
Fault injection model	Random bit flips uniformly distributed across the full codeword
Algorithms evaluated	RS(12,4), C-RRNS (MRC), 3NRM-RRNS (MLD), 2NRM-RRNS (MLD)

The fault injection model in the MATLAB simulation applies random bit flips uniformly across the entire codeword, including any zero-padding bits. This differs from the FPGA implementation described in Section 3.2.2, where fault injection is strictly confined to the W_{valid} valid codeword bits. This distinction is important when comparing the two sets of results, as discussed in Section 3.1.4.

3.1.2 Decoding Success Rate Comparison

Figure 3.2 shows the average decoding success rate as a function of fault rate for all four algorithms, with C-RRNS using the Mixed Radix Conversion (MRC) decoder (reproduced from

the Interim Report [Interim Report, Figure 2]).



> **Figure 3.2** Average decoding success rate vs. fault rate for RS, C-RRNS (MRC), 3NRM-RRNS (MLD), and 2NRM-RRNS (MLD). *(Reproduced from Interim Report, Figure 2.)*

The key findings from this simulation are as follows:

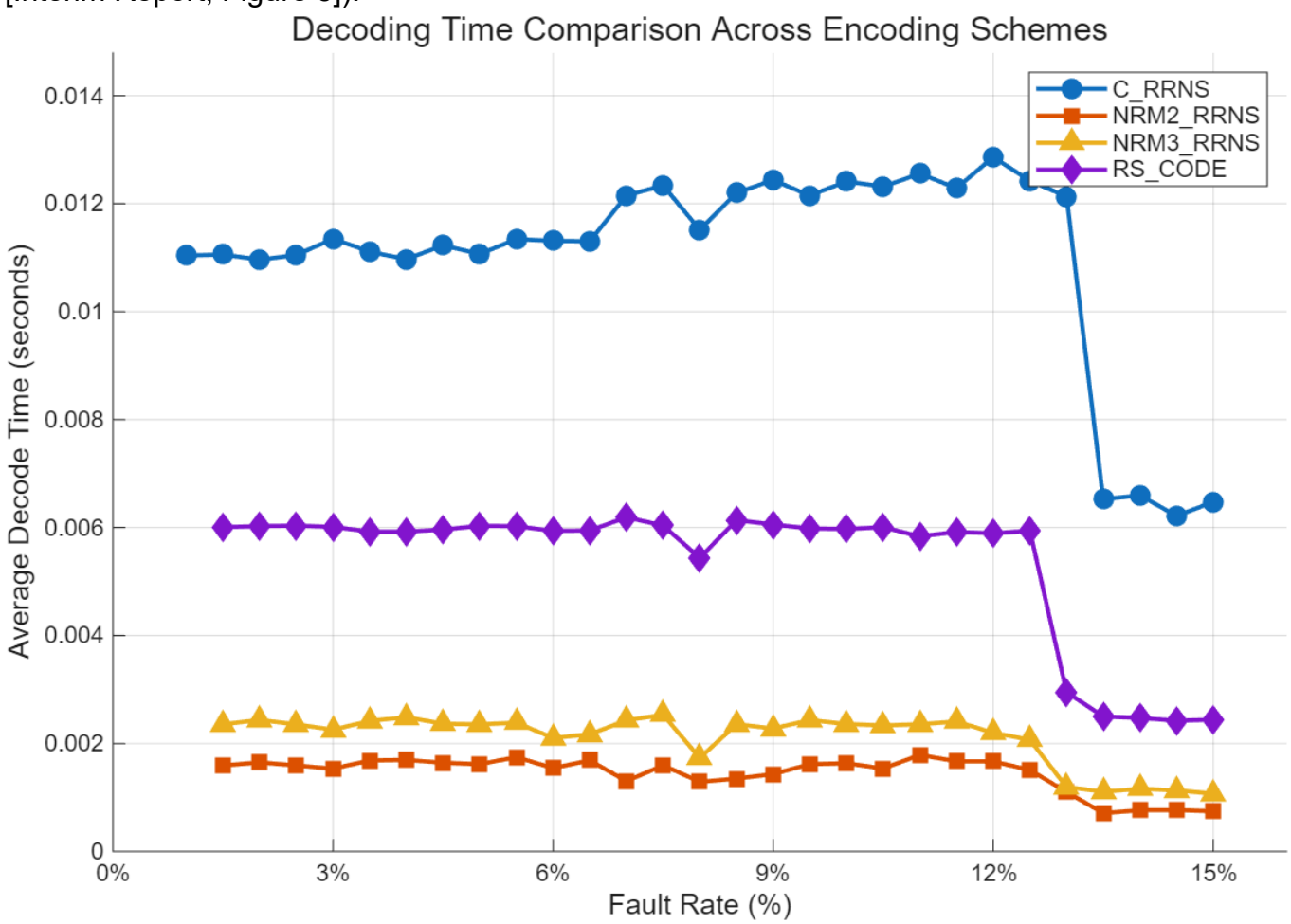
1. **RS(12,4) exhibits the highest robustness at elevated fault rates.** The RS code maintains a 100% decoding success rate up to approximately 9.5% fault rate, after which it degrades rapidly. Even at 14% fault rate, it retains approximately 6–8% success, demonstrating strong resilience in high-error environments.
2. **C-RRNS (MRC) performs well at low fault rates but degrades rapidly.** C-RRNS achieves 100% success below approximately 6% fault rate. Beyond this threshold, performance drops sharply, falling below 20% at 12%. This behaviour is consistent with the MRC decoder's lack of error correction capability — it reconstructs data directly from the three non-redundant residues without exploiting the redundant moduli.
3. **3NRM-RRNS shows stronger resilience than 2NRM-RRNS.** With t=3 correction capability, 3NRM-RRNS sustains over 60% success until approximately 11% fault rate, then gradually declines. 2NRM-RRNS, with t=2 correction capability, stabilises at approximately 35–37% success between 8–11% fault rate.
4. **Maximum fault rate for 100% success** (Table 3.2):

Table 3.2 Maximum fault rate achieving 100% decoding success across 1,000 trials (MATLAB simulation).

Algorithm	Max fault rate for 100% success	Error correction capability
RS(12,4)	~9.5%	t=4 symbols
C-RRNS (MRC)	~6%	None (direct reconstruction)
3NRM-RRNS (MLD)	~7%	t=3 residues
2NRM-RRNS (MLD)	~6%	t=2 residues

3.1.3 Decoding Latency Comparison

Figure 3.3 shows the average decoding latency (measured in MATLAB execution time) for all four algorithms, with C-RRNS using the MRC decoder (reproduced from the Interim Report [Interim Report, Figure 5]).



> **Figure 3.3** Average decoding latency (MATLAB seconds) vs. fault rate for RS, C-RRNS (MRC), 3NRM-RRNS (MLD), and 2NRM-RRNS (MLD). *(Reproduced from Interim Report,*

> *Figure 5.)*

The key findings are:

1. **C-RRNS (MRC) exhibits significantly higher latency** than all other algorithms, particularly at low fault rates, due to the exhaustive subset enumeration and majority voting in the MRC decoding procedure. Latency reaches approximately 10–12 ms at low fault rates.
2. **2NRM-RRNS, 3NRM-RRNS, and RS exhibit much faster and more stable decoding times**, with RS showing minimal variation across fault rates. This highlights the computational efficiency of the MLD-based RRNS decoders relative to the MRC approach.
3. **The latency ranking** (fastest to slowest) is: RS ≈ 2NRM-RRNS ≈ 3NRM-RRNS ≪ C-RRNS (MRC). This is consistent with the FPGA hardware results presented in Section 4.5, where C-RRNS-MLD (928 cycles) is the slowest correcting decoder and 2NRM-RRNS Parallel (24 cycles) is the fastest.

3.1.4 Key Conclusions and Comparison with FPGA Results

The MATLAB simulation phase established the following conclusions, which motivated the subsequent FPGA implementation:

1. **Feasibility confirmed**: All four algorithms provide satisfactory error correction within their respective correction capability limits. The 3NRM-RRNS and 2NRM-RRNS algorithms proposed in [1] are viable ECC schemes for hybrid CMOS/non-CMOS memories.
2. **Storage efficiency advantage of 2NRM-RRNS**: With a 41-bit codeword (vs. 48 bits for RS and 3NRM-RRNS, and 61 bits for C-RRNS), 2NRM-RRNS achieves the highest storage efficiency, requiring 14.6% fewer bits than RS and 32.8% fewer bits than C-RRNS.
3. **Recommended algorithm**: Considering both fault tolerance and storage efficiency, 2NRM-RRNS is the recommended ECC algorithm for hybrid CMOS/non-CMOS memory systems.

Comparison with FPGA hardware results:

The FPGA implementation results (Chapter 4) are broadly consistent with the MATLAB simulation conclusions, but with two notable differences attributable to differences in the fault injection model and decoder implementation:

- **C-RRNS-MLD achieves 100% success in FPGA but not in MATLAB.** In the MATLAB simulation, C-RRNS uses the MRC decoder (no error correction), which explains its degradation above 6% fault rate. In the FPGA implementation, C-RRNS uses the MLD decoder (t=3 correction), which achieves 100% success across the full 0–10% BER range. This is the theoretically expected result for MLD with t=3 correction under random single-bit injection.

- **Fault injection model difference.** The MATLAB simulation injects faults uniformly across the full codeword (including zero-padding bits), while the FPGA implementation injects faults strictly within the W_{valid} valid bits. This means the effective fault rate experienced by the decoder is slightly lower in the FPGA case, contributing to the improved success rates observed in the hardware results.
- **The relative ranking of algorithms is consistent** between MATLAB and FPGA: RS and C-RRNS-MLD provide the highest fault tolerance, followed by 3NRM-RRNS, then 2NRM-RRNS. The storage efficiency ranking (2NRM-RRNS best, C-RRNS worst) is identical in both platforms.

3.2 FPGA Implementation Phase

3.2.1 Top-Level System Architecture

3.2.1.1 Overview

The FPGA-based fault-tolerance evaluation platform is designed around a **master-slave architecture**, in which a PC host acts as the high-level controller responsible for test configuration and result visualisation, while the FPGA target autonomously executes the full BER sweep and returns a consolidated result packet upon completion. The two sides communicate exclusively through a **UART serial link** operating at 921,600 bps. Figure 3.4 illustrates the top-level system topology.

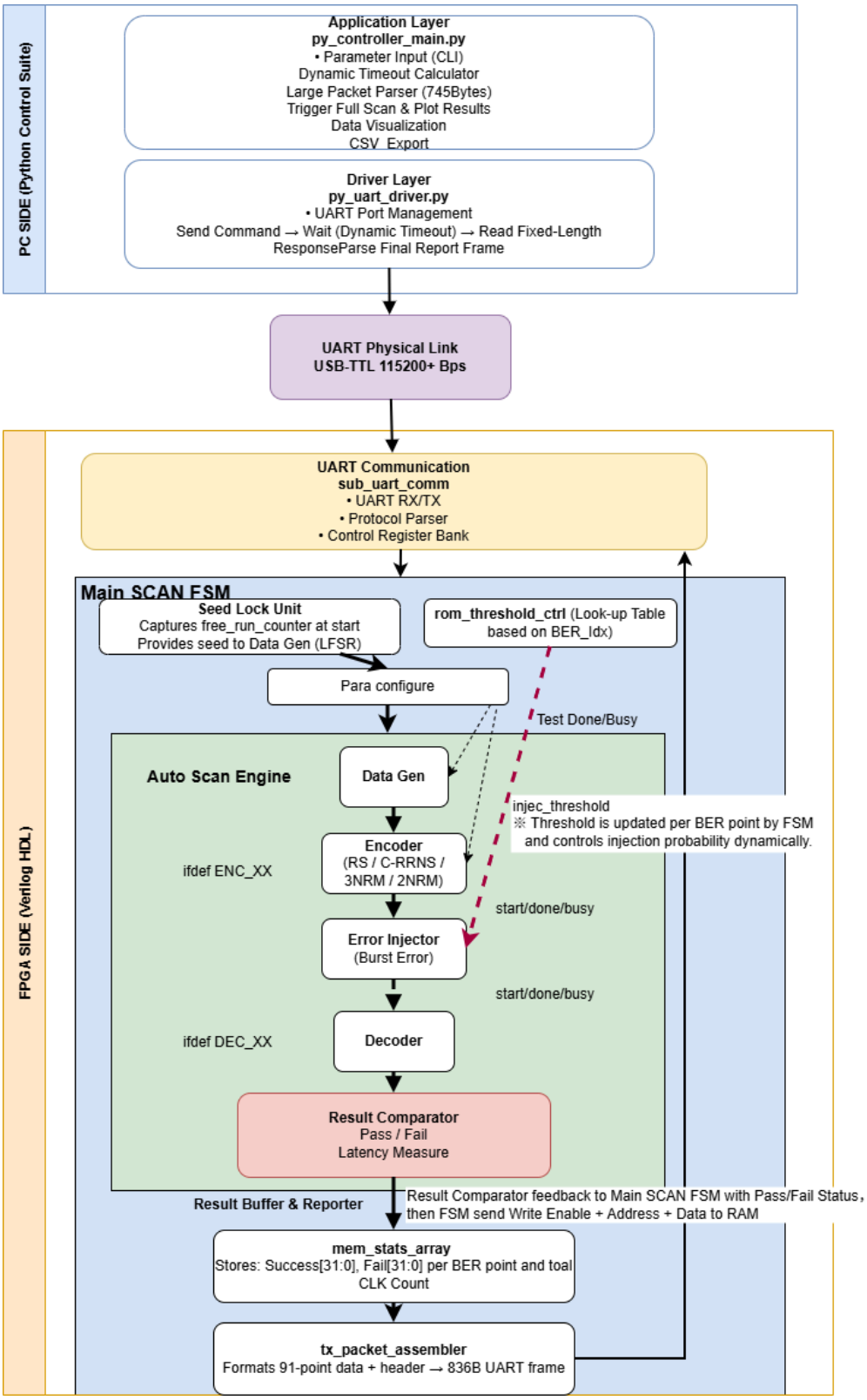


Figure 3.4 Top-level system architecture of the FPGA fault-tolerance evaluation platform (Artix-7 xc7a100t, Arty A7-100T development board).

A key design principle adopted throughout this work is the **Single-Algorithm-Build** strategy: each Vivado synthesis run instantiates exactly one codec algorithm, selected at compile time via a Verilog preprocessor macro (`BUILD_ALGO_xxx`). This ensures that the resource utilisation figures reported for each algorithm are free from cross-algorithm interference, yielding accurate

and directly comparable LUT, flip-flop, DSP, and Block RAM counts. Switching between algorithms requires only a one-line change in the header file `src/interfaces/main_scan_fsm.vh`, followed by a full re-synthesis.

3.2.1.2 Hierarchical Module Structure

The system is organised into two major domains — a PC-side Python application and an FPGA-side Verilog implementation — as shown in the hierarchical tree below.

```
System Architecture
|
├── [PC Side]  (Python host application)
|   ├── py_controller_main.py        - Main controller: user CLI, command
framing,
|   |                                   result parsing, CSV export, auto-
plotting
|   ├── gen_rom.py                   - Offline ROM generator: pre-computes
10,605
|   |                                   BER threshold entries →
threshold_table.coe
|   |                                   and 8,192 error-pattern entries →
error_lut.coe
|   └── Visualisation scripts
|       ├── compare_ber_curves.py    - Multi-algorithm BER success-rate
comparison
|       ├── plot_utilisation.py      - FPGA resource utilisation comparison
|       ├── plot_latency.py          - Encoder/decoder latency comparison
|       └── plot_storage_efficiency.py - Codeword storage overhead comparison
|
└── [FPGA Side]  (Verilog / SystemVerilog, Xilinx Artix-7 xc7a100t)
    |
    ├── UART Communication Layer
    |   ├── uart_rx_module           - Serial-to-parallel byte receiver (16×
oversampling)
    |   ├── uart_tx_module           - Parallel-to-serial byte transmitter
    |   ├── protocol_parser          - Frame synchronisation, XOR checksum
verification,
    |   |                              parameter extraction from 12-byte
command frame
    |   └── ctrl_register_bank       - Atomic parameter latch; auto-asserts
test_active
    |                                  upon receipt of a valid configuration
frame
```

```
    │
    ├── Seed Lock Unit               - Captures free-running 32-bit counter at
test start;
    │                                  holds seed constant across all 101 BER
points to
    │                                  ensure statistical consistency
    │
    ├── Main Scan FSM                - Top-level test orchestrator; iterates
over 101 BER
    │   │                              points (0 % to 10 %, step 0.1 %),
invokes Auto Scan
    │   │                              Engine for each point, accumulates
statistics, and
    │   │                              triggers the result upload phase
    │   └── rom_threshold_ctrl       - Retrieves the 32-bit LFSR injection
threshold from
    │                                  Block RAM (threshold_table.coe, depth
10,605 entries)
    │
    ├── Auto Scan Engine             - Single-trial execution core
    │   ├── PRBS Generator (LFSR)    - 32-bit Galois LFSR, seeded by Seed Lock
Unit
    │   ├── Encoder Wrapper          - Compile-time algorithm selection;
routes to the
    │   │                              active encoder (2NRM / 3NRM / C-RRNS /
RS)
    │   ├── Error Injector Unit      - ROM-based burst/random-bit fault
injection;
    │   │                              error patterns pre-computed in
error_lut.coe
    │   ├── Decoder Wrapper          - Compile-time algorithm selection;
routes to the
    │   │                              active decoder
    │   └── Result Comparator        - Compares original vs. decoded symbol;
records
    │                                  pass/fail, actual flip count, encoder
latency,
    │                                  and decoder latency in clock cycles
    │
    └── Result Buffer & Reporter
        ├── mem_stats_array          - True dual-port Block RAM; stores 101 ×
30-byte
        │                              per-point statistics (240-bit entries,
RAMB36)
        └── tx_packet_assembler      - Serialises BRAM contents into a 3,039-
```

```
byte
                                                UART response frame with XOR checksum
```

3.2.1.3 System Workflow

The overall test procedure is divided into three sequential phases, as described below.

Phase 1 — Configuration

The user launches `py_controller_main.py` on the PC and specifies four parameters via a command-line interface:

Parameter	Description	Range
`Algo_ID`	Algorithm under test	0–6 (see Table 3.1)
`Error_Mode`	Fault injection mode	0 = random single-bit, 1 = cluster burst
`Burst_Len`	Burst length *L*	1–15 bits
`Sample_Count`	Trials per BER point	1–1,000,000

Table 3.3 Algorithm ID mapping.

Algo_ID	Algorithm	Decoder Architecture	Codeword (bits)
0	2NRM-RRNS (Parallel)	15-channel parallel MLD	41
1	3NRM-RRNS	Sequential FSM MLD (84 triplets)	48
2	C-RRNS-MLD	Sequential FSM MLD (84 triplets)	61
3	C-RRNS-MRC	Direct Mixed Radix Conversion	61
4	C-RRNS-CRT	Direct Chinese Remainder Theorem	61
5	RS(12,4)	Berlekamp–Massey + Chien + Forney	48
6	2NRM-RRNS (Serial)	Sequential FSM MLD (15 pairs)	41

The host assembles a compact **12-byte downlink command frame** structured as:

```
[Header: 0xAA 0x55] [CmdID: 0x01] [Length: 0x07] [Burst_Len] [Algo_ID]
```

```
[Error_Mode] [Sample_Count: 4 bytes, Big-Endian] [XOR Checksum]
```

and transmits it over UART. Receipt of this frame by the FPGA implicitly triggers the start of the test sweep — no separate start command is required. This "configuration-as-trigger" protocol minimises communication overhead and eliminates the risk of a race condition between configuration and start signals.

Phase 2 — Autonomous BER Sweep

Upon receiving a valid configuration frame, the FPGA performs the following sequence entirely without further PC intervention:

1. **Parameter latch**: `ctrl_register_bank` atomically captures all four parameters and asserts `test_active` in the same clock cycle, preventing any partial-update hazard.
2. **Seed capture**: `seed_lock_unit` samples the free-running 32-bit counter one cycle after `test_active` is asserted and holds the seed constant for the entire sweep. This ensures that all 101 BER points are driven by the same pseudo-random sequence, making the results statistically consistent and reproducible.
3. **101-point loop**: `Main Scan FSM` iterates `ber_idx` from 0 to 100, corresponding to target BER values of 0.0 % to 10.0 % in steps of 0.1 %. At each point:
   - The 32-bit LFSR injection threshold is retrieved from `rom_threshold_ctrl`. This threshold was pre-computed offline by `gen_rom.py` using the formula $T = \lfloor P_{\text{trigger}} \times (2^{32} - 1) \rfloor$, where $P_{\text{trigger}} = \text{BER}_{\text{target}} \times W_{\text{valid}} / L$, and W_{valid} is the algorithm-specific valid codeword width.
   - `Auto Scan Engine` executes *N* independent trials (where *N* = `Sample_Count`). In each trial: a 16-bit pseudo-random symbol is generated → encoded into a codeword → faults are injected according to the current threshold and burst length → the corrupted codeword is decoded → the decoded symbol is compared against the original. Pass/fail outcome, actual flip count, and encoder/decoder clock cycles are accumulated into running totals.
   - Upon completion of *N* trials, the aggregated statistics for the current BER point are written to `mem_stats_array`.
4. On completion of all 101 points, `test_active` is de-asserted and the upload phase is triggered.

Phase 3 — Result Upload

`tx_packet_assembler` reads all 101 entries from `mem_stats_array` and serialises them into a **3,039-byte uplink response frame**:

```
Header(2) + CmdID(1) + Length(2) + GlobalInfo(3) + 101 × PerPointData(30) +
Checksum(1)
```

```
= 3,039 bytes total
```

Each 30-byte per-point record contains: BER index (1 B), success count (4 B), fail count (4 B), actual flip count (4 B), total clock count (8 B), encoder clock count (4 B), decoder clock count (4 B), and one reserved byte. The PC receives the frame, verifies the XOR checksum, parses all 101 records, and exports the results to a timestamped CSV file. Visualisation scripts are then invoked automatically to generate the comparison plots presented in Chapter 4.

3.2.1.4 Main Scan FSM State Diagram

The top-level control flow of the FPGA is governed by `Main Scan FSM`, whose state transitions are shown in Figure 3.5.

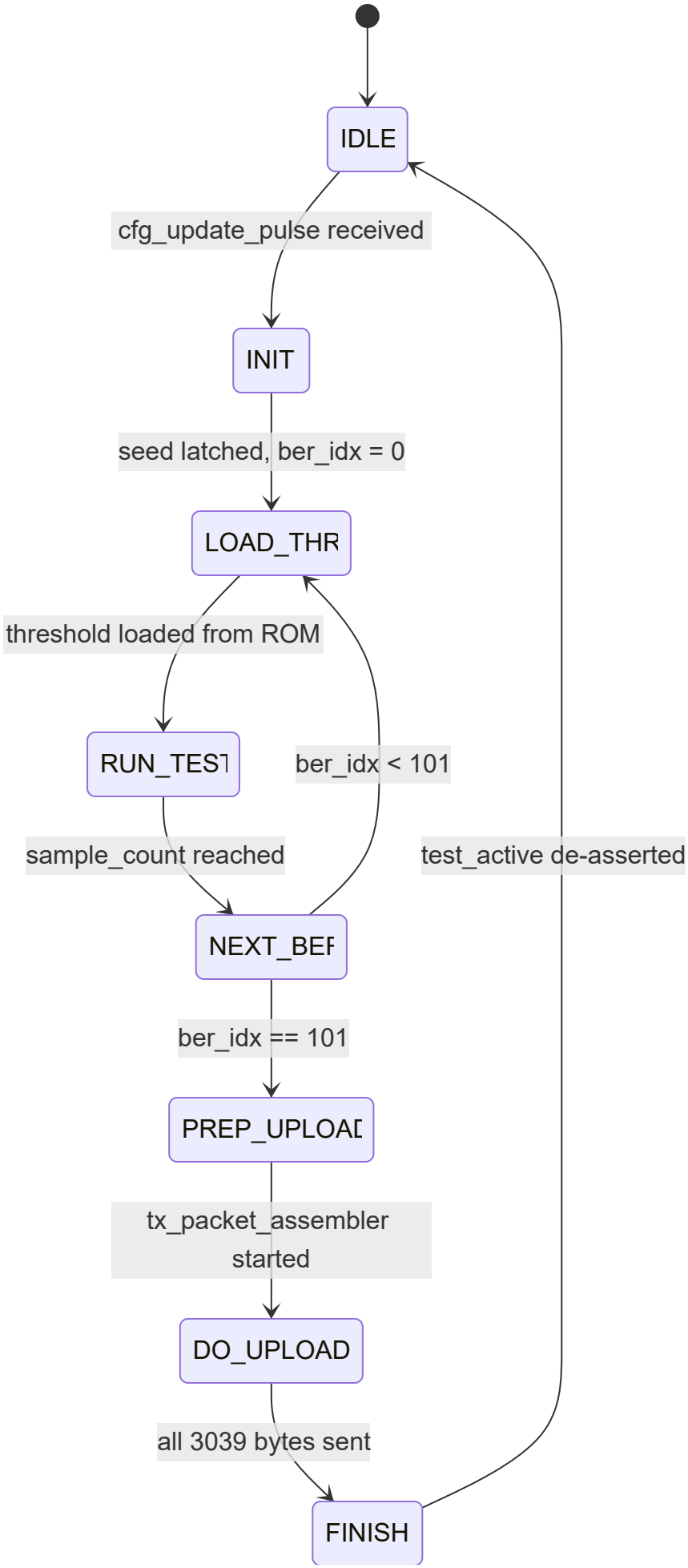

IDLE
cfg_update_pulse received
INIT
seed latched, ber_idx = 0
LOAD_THR
threshold loaded from ROM
RUN_TEST
ber_idx < 101
sample_count reached
test_active de-asserted
NEXT_BER
ber_idx == 101
PREP_UPLOAD
tx_packet_assembler started
DO_UPLOAD
all 3039 bytes sent
FINISH

Figure 3.5 State diagram of the Main Scan FSM.

The FSM employs an **edge-triggered start mechanism**: an internal rising-edge detector on `test_active` generates a single-cycle `sys_start_pulse`, preventing re-triggering if the signal remains asserted. A hardware abort button (mapped to FPGA pin B9, debounced over 16 ms at 100 MHz) asserts `sys_abort` with the highest priority, forcing an immediate return to `IDLE` from any state. This provides a reliable mechanism to interrupt a long-running test without requiring a full FPGA reset.

3.2.1.5 Clock Domain and Reset Strategy

The entire design operates within a **single clock domain** driven by the 100 MHz on-board oscillator of the Arty A7-100T development board. This eliminates the need for asynchronous FIFOs or clock-domain crossing synchronisers, simplifying timing closure and reducing resource overhead.

The UART baud-rate generator uses an integer divider of 109, yielding an actual baud rate of 917,431 bps — a deviation of −0.45 % from the target 921,600 bps. This is well within the ±2.5 % tolerance of standard UART receivers (based on 16× oversampling with the sampling point at the bit centre), and has been verified to produce no framing errors over the 3,039-byte response frame.

All registers adopt an **asynchronous-assert, synchronous-release** reset strategy. The external reset signal (`rst_n`) is passed through a two-stage flip-flop synchroniser before being distributed as `sys_rst_n` to all sub-modules. This prevents metastability at power-on while ensuring deterministic release timing aligned to the rising edge of the system clock, in accordance with standard FPGA design practice for Xilinx Artix-7 devices.

3.2.2 Probabilistic Fault Injection Engine

3.2.2.1 LFSR-Based Pseudo-Random Fault Generation

The fault injection subsystem is built around a **32-bit Galois Linear Feedback Shift Register (LFSR)**, which serves as the sole source of pseudo-randomness for both injection triggering and error-position selection. The Galois configuration was chosen over the more common Fibonacci topology because it distributes the feedback XOR operations across individual flip-flops, enabling a single-cycle update with minimal combinational depth — a critical requirement for maintaining 100 MHz timing closure.

At each rising clock edge, the LFSR advances by one step, producing a new 32-bit pseudo-random value. This output is partitioned and reused for two independent purposes without

requiring additional LFSR instances:

- **Bits [31:0]** are compared against the 32-bit injection threshold to determine whether a fault should be injected in the current trial.
- **Bits [5:0]** are used directly as the random offset into the error-pattern look-up table, selecting the starting bit position of the injected burst.

This dual-use strategy eliminates the need for a second random source, reduces hardware resource consumption, and ensures that the injection decision and the error position are determined within the same clock cycle.

Algorithm 3.1: 32-bit Galois LFSR — Single-Cycle Update and Dual-Use Output

```
Algorithm 3.1: 32-bit Galois LFSR Update
──────────────────────────────────────────────────────────────────

Polynomial: x^32 + x^22 + x^2 + x + 1  (taps at bits [31, 21, 1, 0])
Period:     2^32 - 1 = 4,294,967,295 cycles

Inputs:
  lfsr_q[31:0]   -- current LFSR state (must be non-zero)
  load_seed      -- 1: load new seed; 0: advance LFSR
  seed_in[31:0]  -- seed value (forced to 1 if zero)

Outputs:
  lfsr_next[31:0]  -- next LFSR state
  inject_trigger   -- 1 if fault should be injected this cycle
  offset[5:0]      -- random bit position for error mask lookup

Step 1 -- Galois right-shift update (combinational, single cycle):
  feedback         := lfsr_q[0]                    // outgoing LSB
  lfsr_next[31]    := feedback                      // tap 31: shift-in
  lfsr_next[30:22] := lfsr_q[31:23]                // plain shift (9 bits)
  lfsr_next[21]    := lfsr_q[22] XOR feedback      // tap 21: XOR
  lfsr_next[20:2]  := lfsr_q[21:3]                 // plain shift (19 bits)
  lfsr_next[1]     := lfsr_q[2]  XOR feedback      // tap 1: XOR
  lfsr_next[0]     := lfsr_q[1]  XOR feedback      // tap 0: XOR

Step 2 -- Sequential register update (on rising clock edge):
  if load_seed:
    lfsr_q := (seed_in != 0) ? seed_in : 1         // zero-seed guard
  else:
    lfsr_q := lfsr_next

Step 3 -- Dual-use output extraction (combinational):
  inject_trigger := (lfsr_next[31:0] < T)           // compare full 32-bit
```

```
value
                                                          // against threshold T
  offset[5:0]    := lfsr_next[5:0]                       // lower 6 bits -> error
position
──────────────────────────────────────────────────────────────────────────────
```

The Galois LFSR update (Step 1) is a purely combinational operation: the next state is computed in a single clock cycle by XOR-ing the feedback bit (outgoing LSB) into the tap positions [21, 1, 0] while shifting all other bits right by one position. This single-cycle update is the key advantage of the Galois topology over the Fibonacci topology, which would require a multi-stage XOR chain. The injection decision (Step 3) uses the full 32-bit output for maximum resolution, while the error position uses only the lower 6 bits, ensuring that both decisions are made within the same clock cycle with no additional latency.

3.2.2.2 Seed Initialisation Mechanism

A key design requirement is that each test run should produce statistically independent results, while all 101 BER points within a single run must be driven by the same pseudo-random sequence to ensure cross-point consistency. This is achieved through a **task-level seed locking** mechanism implemented in the `seed_lock_unit` module.

Rather than receiving a seed from the PC host, the FPGA captures its own seed autonomously. A 32-bit free-running counter increments continuously at 100 MHz. At the moment the FPGA receives a valid configuration frame from the PC — specifically, one clock cycle after `test_active` is asserted — the current counter value is latched as the LFSR seed. Since the exact timing of the PC command depends on the user's interaction and the USB-UART bridge latency, the captured value is effectively unpredictable, providing natural randomness between successive test runs.

Once latched, the seed is held constant throughout the entire 101-point sweep. The `lock_en` signal remains asserted from the `INIT` state until the `FINISH` state of the Main Scan FSM, preventing any re-capture. A zero-seed guard is also implemented: if the captured counter value is zero, the seed is forced to 1, preventing the LFSR from entering the all-zero lock-up state.

This mechanism offers two important advantages. First, it eliminates the need for the PC to transmit a seed value, simplifying the communication protocol. Second, it guarantees that the statistical properties of the pseudo-random sequence are identical across all 101 BER points within a single test run, making the BER sweep results directly comparable.

3.2.2.3 Injection Trigger Mechanism

The target BER is realised through a **probabilistic threshold comparison** rather than a deterministic injection schedule. This approach is a novel contribution of this work, as it decouples the injection probability from the number of test samples, enabling the sample count to be configured freely over a wide range (1 to 1,000,000) without any change to the hardware or the pre-computed ROM tables.

BER Definition

The BER in this system is defined as the ratio of the total number of injected bit flips to the total number of valid codeword bits processed:

$$\mathrm{BER}_{\mathrm{target}} = \frac{N_{\mathrm{flips}}}{N_{\mathrm{samples}} \times W_{\mathrm{valid}}}$$

where W_{valid} is the algorithm-specific valid codeword width (e.g., 41 bits for 2NRM-RRNS, 61 bits for C-RRNS), and L is the burst length. This definition ensures that algorithms with different codeword lengths are evaluated under a fair and comparable injection intensity.

Threshold Calculation

For a given target BER, burst length L, and algorithm codeword width W_{valid}, the per-trial injection probability is:

$$P_{\mathrm{trigger}} = \frac{\mathrm{BER}_{\mathrm{target}} \times W_{\mathrm{valid}}}{L}$$

This probability is mapped to a 32-bit unsigned integer threshold:

$$T = \left\lfloor P_{\mathrm{trigger}} \times (2^{32} - 1) \right\rfloor$$

At each trial, the 32-bit LFSR output R is compared against T. If $R < T$, a fault is injected; otherwise, the codeword passes through unmodified. Since the LFSR output is uniformly distributed over $[0, 2^{32} - 1]$, the long-run injection rate converges to P_{trigger} by the law of large numbers.

Offline Pre-computation

All threshold values for the 101 BER points (0.0 % to 10.0 %, step 0.1 %), 7 algorithms, and 15 burst lengths are pre-computed offline by the Python script `gen_rom.py` and stored in a Block RAM initialised from `threshold_table.coe` (10,605 entries, 32 bits wide). During the test sweep, the FPGA retrieves the appropriate threshold in a single clock cycle via the `rom_threshold_ctrl` module, with no floating-point arithmetic required at runtime.

3.2.2.4 Error Pattern Generation and Boundary Safety

Once the injection trigger fires, the specific bit positions to be flipped are determined by a **ROM-based look-up table** (`error_lut.coe`). This approach replaces the conventional dynamic barrel-shifter method, which would require a long combinational path and risk timing violations at 100 MHz.

The error look-up table has a depth of 8,192 entries and a width of 64 bits. Each entry stores a pre-computed 64-bit error mask with exactly L bits set to 1, starting at a specific offset within the valid codeword region. The table is addressed by a 13-bit index formed by concatenating three fields:

$$\text{Address} = \{\text{algo_id}[2:0],\ (\text{burst_len} - 1)[3:0],\ \text{random_offset}[5:0]\}$$

The `random_offset` field is taken directly from the lower 6 bits of the LFSR output, providing 64 possible starting positions for each burst length and algorithm combination.

Boundary Safety

A critical constraint is that the injected burst must fall entirely within the W_{valid} valid bits of the codeword, never touching the zero-padding bits in the upper portion of the 64-bit bus. This constraint is enforced entirely at the offline pre-computation stage: for any address where `random_offset` > $W_{\text{valid}} - L$, the corresponding ROM entry is set to zero (no injection). As a result, the FPGA hardware requires no boundary-checking logic whatsoever — any out-of-range LFSR output simply produces a zero mask, which has no effect on the codeword. This fail-safe mechanism guarantees single-cycle injection latency and eliminates a class of potential timing violations.

Random Single-Bit Mode

When `burst_len` = 1, the injection reduces to a single random bit flip. The valid offset range is $[0, W_{\text{valid}} - 1]$, and the 64 possible LFSR offsets are mapped such that offsets 0 to $W_{\text{valid}} - 1$ produce a valid single-bit mask, while offsets W_{valid} to 63 produce a zero mask (no injection). The effective injection probability is therefore scaled by a factor of $W_{\text{valid}}/64$, which is compensated in the threshold calculation as described in Section 3.2.2.3.

Cluster Burst Mode

When `burst_len` = $L > 1$, the valid offset range shrinks to $[0, W_{\text{valid}} - L]$, ensuring the entire L -bit burst fits within the codeword. The number of valid offsets is $W_{\text{valid}} - L + 1$, and the compensation factor applied to the threshold becomes $64/(W_{\text{valid}} - L + 1)$.

3.2.2.5 Impact of Burst Length on Maximum Achievable BER

The probabilistic injection mechanism imposes a fundamental upper bound on the achievable BER for a given burst length and algorithm. Since the injection probability P_{trigger} cannot exceed 1, the maximum achievable target BER is:

$$\text{BER}_{\text{max}} = \frac{L}{W_{\text{valid}}}$$

This bound arises because, even if every trial triggers an injection ($P_{\text{trigger}} = 1$), the number of flipped bits per trial is exactly L, and the total valid bits per trial is W_{valid}.

As the burst length L increases, BER_{max} increases proportionally. However, the number of valid injection offsets simultaneously decreases, which reduces the effective randomness of the error position. For very large L relative to W_{valid}, the injected burst covers a significant fraction of the codeword, and the error pattern becomes less representative of realistic channel conditions. In this work, burst lengths up to $L = 15$ are supported, which is sufficient to model the cluster fault patterns observed in hybrid CMOS/non-CMOS memory technologies.

The following table summarises the maximum achievable BER for each algorithm at selected burst lengths.

Table 3.4 Maximum achievable BER ($\text{BER}_{\text{max}} = L/W_{\text{valid}}$) for each algorithm and selected burst lengths.

Algorithm	W_{valid} **(bits)**	$L = 1$	$L = 5$	$L = 8$	$L = 15$
2NRM-RRNS	41	2.4 %	12.2 %	19.5 %	36.6 %
3NRM-RRNS	48	2.1 %	10.4 %	16.7 %	31.3 %
C-RRNS	61	1.6 %	8.2 %	13.1 %	24.6 %
RS(12,4)	48	2.1 %	10.4 %	16.7 %	31.3 %

Since the target BER sweep in this work covers 0 % to 10 %, all four algorithms can be fully evaluated under both random single-bit ($L = 1$) and cluster burst ($L = 5$, $L = 8$) injection modes without exceeding the theoretical maximum.

3.2.2.6 Design Advantages and Novel Contributions

The probabilistic fault injection engine described in this section represents a **novel contribution** of this work, offering several advantages over conventional approaches:

1. **Minimal memory footprint**: The entire injection subsystem requires only two Block RAMs — `threshold_table.coe` (10,605 × 32-bit entries, approximately 41 KB) and

`error_lut.coe` (8,192 × 64-bit entries, approximately 64 KB). This is orders of magnitude smaller than a pre-generated fault sequence stored in memory.

2. **Arbitrary sample count**: Because injection decisions are made independently at each trial using a probabilistic comparison, the sample count per BER point can be set to any value between 1 and 1,000,000 without modifying the hardware or the ROM tables. This flexibility allows the user to trade off test duration against statistical confidence.
3. **Statistical rigour**: In this work, 100,000 samples are collected per BER point. By the law of large numbers, the actual injection rate converges to the target BER with a standard deviation of approximately $\sqrt{P(1-P)/N} \approx 0.16\%$ at $P = 5\%$, $N = 100{,}000$. This level of statistical precision is sufficient to distinguish the performance differences between the evaluated algorithms.
4. **Single-cycle injection latency**: The ROM look-up table approach eliminates all runtime arithmetic, producing the 64-bit error mask in a single clock cycle. This ensures that the injection subsystem does not introduce any pipeline stalls or timing violations at 100 MHz.
5. **Algorithm-agnostic design**: The injection engine is parameterised by `algo_id`, `burst_len`, and `random_offset`, and operates identically regardless of which ECC algorithm is under test. Adding a new algorithm requires only updating the ROM tables offline — no changes to the FPGA injection logic are needed.

3.2.3 End-to-End Test Loop and Algorithm Extensibility

3.2.3.1 Single-Trial Execution Pipeline

Each test trial is executed by the `Auto Scan Engine` module as a deterministic five-stage pipeline. The stages proceed sequentially within a single invocation of the engine, with the FSM advancing through each stage upon completion of the previous one.

Stage 1 — Data Generation

A 16-bit pseudo-random test symbol is extracted from the current LFSR output. The symbol is derived from the lower 16 bits of the 32-bit LFSR register, ensuring that the test data is statistically independent from the injection trigger decision (which uses the full 32-bit value).

Stage 2 — Encoding

The test symbol is passed to the `encoder_wrapper`, which routes it to the active encoder module selected at compile time. The encoder produces a codeword of algorithm-specific width (41 to 61 bits), right-aligned within a 64-bit bus with zero-padding in the upper bits. The encoder asserts a `done` signal upon completion; the FSM waits for this signal before proceeding. The

encoder latency in clock cycles is measured from the assertion of `start` to the assertion of `done`, and is accumulated for statistical reporting.

Stage 3 — Fault Injection

The encoded codeword is passed to the `error_injector_unit`. Based on the current LFSR output and the pre-computed error look-up table, the injector either applies a burst error mask (XOR operation) or passes the codeword through unmodified, as described in Section 3.2.2. The actual number of flipped bits is recorded for statistical reporting.

Stage 4 — Decoding

The (potentially corrupted) codeword is passed to the `decoder_wrapper`, which routes it to the active decoder module. The decoder asserts a `valid` signal when the decoded symbol is ready, along with an `uncorrectable` flag if the error pattern exceeds the algorithm's correction capability. The FSM waits for `valid` before proceeding. The decoder latency in clock cycles is measured from the assertion of `start` to the assertion of `valid`, and is accumulated separately from the encoder latency.

Stage 5 — Comparison and Statistics

The `result_comparator` compares the decoded symbol against the original test symbol. A trial is counted as a **pass** only if both of the following conditions are satisfied simultaneously: (1) the decoder reports no uncorrectable error, and (2) the decoded symbol is bit-for-bit identical to the original. If either condition fails, the trial is counted as a **failure**. This double-check rule prevents decoder false-positives — cases where the decoder claims success but produces an incorrect output — from inflating the measured success rate.

The following statistics are accumulated for each BER point:

- Success count and failure count
- Total actual flip count (sum of injected bits across all trials)
- Total clock cycles (encoder latency + injection + decoder latency)
- Encoder clock cycles (accumulated separately)
- Decoder clock cycles (accumulated separately)

3.2.3.2 Encoder Wrapper and Decoder Wrapper: Unified Interface for Algorithm Extensibility

A central design goal of this platform is to support multiple ECC algorithms with minimal code duplication and zero changes to the test infrastructure when a new algorithm is added. This is

achieved through the `encoder_wrapper` and `decoder_wrapper` modules, which provide a **unified, algorithm-agnostic interface** to the `Auto Scan Engine`.

Unified Interface

All encoder modules expose the following standardised interface:

Signal	Direction	Description
`clk`, `rst_n`	Input	System clock and active-low reset
`start`	Input	Single-cycle pulse to begin encoding
`data_in_A`	Input	16-bit input symbol
`codeword_out`	Output	Encoded codeword (right-aligned in 64-bit bus)
`done`	Output	Asserted for one cycle when encoding is complete

All decoder modules expose a corresponding standardised interface:

Signal	Direction	Description
`clk`, `rst_n`	Input	System clock and active-low reset
`start`	Input	Single-cycle pulse to begin decoding
`residues_in`	Input	64-bit packed codeword (right-aligned)
`data_out`	Output	Decoded 16-bit symbol
`valid`	Output	Asserted for one cycle when decoding is complete
`uncorrectable`	Output	Asserted if the error pattern exceeds correction capability

Compile-Time Algorithm Selection

The `encoder_wrapper` and `decoder_wrapper` modules use Verilog preprocessor conditionals (`ifdef`) to instantiate exactly one algorithm at compile time. The active algorithm is selected by defining a single macro in the header file `src/interfaces/main_scan_fsm.vh`. All other algorithm branches are compiled out, ensuring that the synthesised netlist contains only the logic for the selected algorithm. This is the foundation of the Single-Algorithm-Build strategy described in Section 3.2.1.

Extensibility

The wrapper architecture makes adding a new ECC algorithm straightforward. The required steps are:

1. Implement the encoder and decoder modules conforming to the standardised interface.
2. Add one `ifdef` branch in `encoder_wrapper.v` and `decoder_wrapper.v`.
3. Add one entry in the algorithm configuration of `gen_rom.py` and regenerate the ROM tables.
4. Add one macro definition in `main_scan_fsm.vh`.

No changes are required to the `Auto Scan Engine`, the Main Scan FSM, the fault injection engine, the result buffer, or the PC-side software. This separation of concerns between the test infrastructure and the algorithm implementation significantly reduces the engineering effort required to evaluate a new algorithm, and is a key design advantage of the platform.

3.2.3.3 Implemented Algorithms and Architectural Variants

The platform was used to implement and evaluate seven algorithm configurations, as summarised in Table 3.5. All seven share the same test infrastructure; only the encoder and decoder modules differ.

Table 3.5 Summary of implemented algorithm configurations.

Algo_ID	Algorithm	Encoder Module	Decoder Module	Decoder Architecture	Codeworc (bits)
0	2NRM-RRNS (Parallel)	`encoder_2nrm`	`decoder_2nrm`	15-channel parallel MLD pipeline	41
1	3NRM-RRNS	`encoder_3nrm`	`decoder_3nrm`	Sequential FSM MLD, 84 triplets	48
2	C-RRNS-MLD	`encoder_crrns`	`decoder_crrns_mld`	Sequential FSM MLD, 84 triplets	61
3	C-RRNS-MRC	`encoder_crrns`	`decoder_crrns_mrc`	Direct Mixed Radix Conversion	61
4	C-RRNS-CRT	`encoder_crrns`	`decoder_crrns_crt`	Direct Chinese Remainder Theorem	61
5	RS(12,4)	`encoder_rs`	`decoder_rs`	Berlekamp–Massey +	48

Algo_ID	Algorithm	Encoder Module	Decoder Module	Decoder Architecture	Codeworc (bits)
				Chien + Forney	
6	2NRM-RRNS (Serial)	`encoder_2nrm`	`decoder_2nrm_serial`	Sequential FSM MLD, 15 pairs × 5 candidates	41

Several observations are worth noting:

C-RRNS variants (IDs 2–4) share the same encoder (`encoder_crrns` , 61-bit codeword) but use three different decoding strategies. C-RRNS-MLD applies Maximum Likelihood Decoding over all C(9,3) = 84 triplets of moduli, providing t = 3 error correction at the cost of approximately 935 clock cycles per decode. C-RRNS-MRC and C-RRNS-CRT use only the three non-redundant moduli for direct reconstruction, achieving decode latencies of approximately 16 and 14 clock cycles respectively, but without error correction capability. This comparison directly illustrates the latency-versus-correction trade-off within the same encoding scheme. Pseudocode descriptions of the C-RRNS encoder and all three decoder variants are provided in Appendix F.

2NRM-RRNS dual implementation (IDs 0 and 6) uses the same encoder (`encoder_2nrm`) and produces identical 41-bit codewords. The two decoders implement the same Maximum Likelihood Decoding algorithm — one using 15 parallel pipeline channels (ID 0) and one using a sequential FSM that iterates through the same 15 modulus pairs, each with up to 5 candidate values (ID 6). Since the decoding algorithm is mathematically identical, the two implementations produce the same BER performance curves, which serves as a cross-validation of correctness. The difference lies entirely in the resource-latency trade-off: the parallel implementation achieves approximately 27 clock cycles at the cost of approximately 22 % LUT utilisation, while the serial implementation requires approximately 225–405 clock cycles but consumes only 2–3 % LUT. This pair of implementations provides a direct, controlled comparison of parallel versus sequential MLD architectures for the same algorithm, which is one of the novel contributions of this work. Pseudocode descriptions of the 2NRM-RRNS encoder (Appendix B), parallel MLD decoder (Appendix C), and serial FSM decoder (Appendix D) are provided in the Appendices. The 3NRM-RRNS encoder and decoder are described in Appendix E, and the RS(12,4) encoder and decoder in Appendix G.

Chapter 4 Results and Discussion

4.1 Experimental Setup Summary

The evaluation platform described in Chapter 3 was used to collect performance data for all seven algorithm configurations under three fault injection scenarios. The test conditions are summarised in Table 4.1.

Table 4.1 Summary of test conditions.

Parameter	Value
Target BER range	0.0 % to 10.0 %, step 0.1 % (101 points)
Samples per BER point	100,000
Fault injection modes	Random single-bit (L=1), Cluster burst L=5, Cluster burst L=8
Algorithms evaluated	2NRM-RRNS (Parallel), 2NRM-RRNS (Serial), 3NRM-RRNS, C-RRNS-MLD, C-RRNS-MRC, C-RRNS-CRT, RS(12,4)
Clock frequency	50 MHz
Target device	Xilinx Artix-7 xc7a100t (Arty A7-100T)

The choice of 100,000 samples per BER point provides a statistical standard deviation of approximately $\sqrt{P(1-P)/N} \approx 0.16\%$ at $P = 5\%$, which is sufficient to distinguish the performance differences between algorithms with high confidence.

4.2 BER Performance Under Random Single-Bit Fault Injection

Figure 4.1 shows the decode success rate as a function of actual injected BER for all seven algorithm configurations under random single-bit fault injection (L=1).

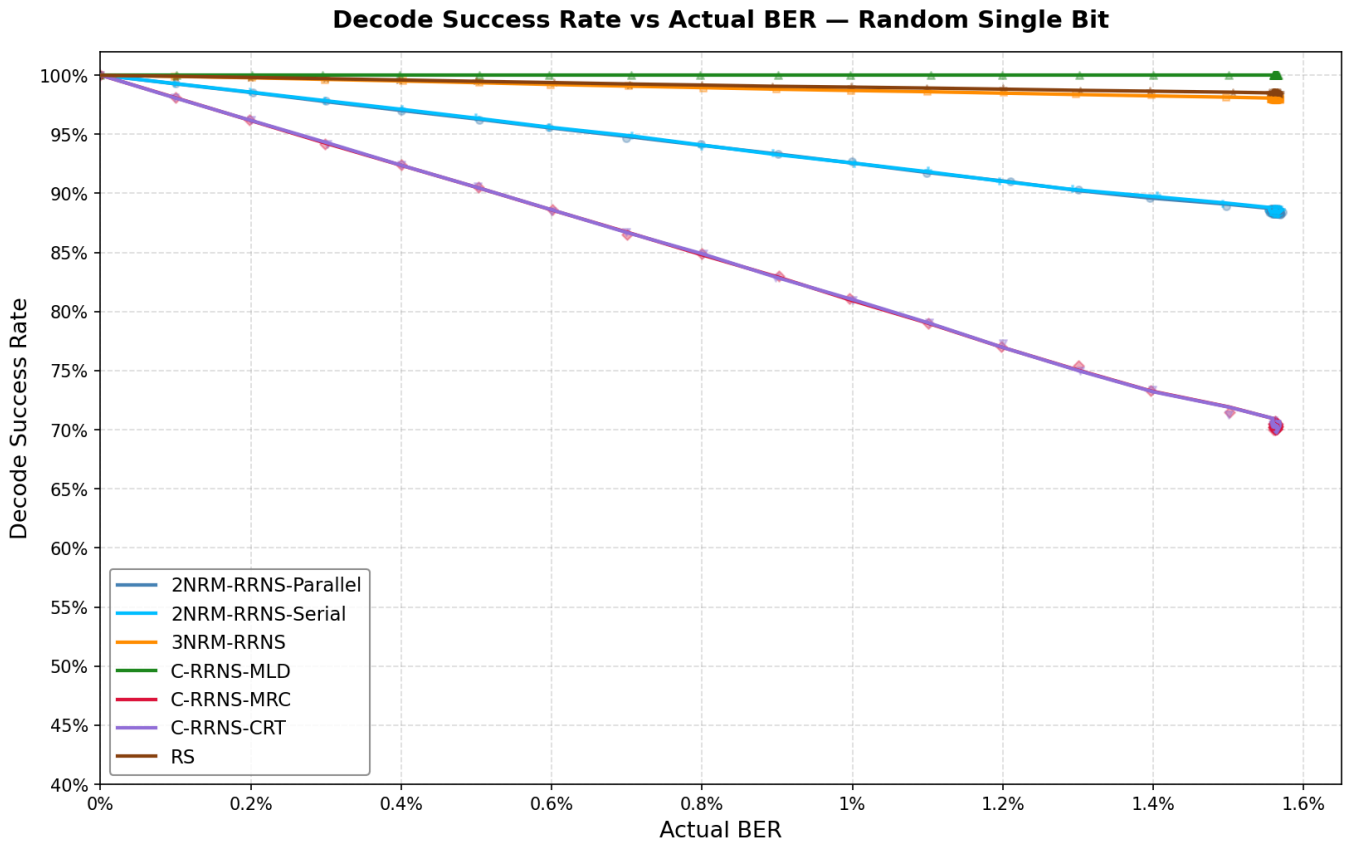


Figure 4.1 Decode success rate vs. actual BER under random single-bit fault injection (L=1, 100,000 samples per BER point).

C-RRNS-MLD: Complete Fault Tolerance

The most significant finding is that **C-RRNS-MLD achieves a 100% decode success rate across the entire tested BER range (0–10%)**. This result is consistent with the theoretical error correction capability of the algorithm. With t=3 correction capability over 9 moduli, and given that a single random bit flip corrupts at most one residue (since each residue occupies a contiguous bit field), the MLD decoder can always recover the original data as long as no more than 3 residues are simultaneously corrupted. Under the test conditions used (BER ≤ 10%, L=1, 100,000 samples), the probability of 4 or more residues being simultaneously corrupted in a single trial is negligibly small. This finding demonstrates that C-RRNS-MLD provides **complete fault tolerance** within the practical BER range of interest for hybrid CMOS/non-CMOS memory applications, and represents a significant result of this work.

2NRM-RRNS: Characteristic Saturation Plateau

The 2NRM-RRNS algorithm (both parallel and serial implementations) exhibits a characteristic two-phase behaviour. In the low-BER region (0–1.6%), the success rate decreases from 100% at BER=0% to approximately 88.4% at BER≈1.6%, following a smooth degradation curve. Beyond this point, the success rate plateaus at approximately 88.4% and remains essentially constant up to BER=10%. This plateau is explained by the t=2 correction limit: once the injected

BER exceeds the threshold at which two or more residues are simultaneously corrupted with non-negligible probability, the decoder's correction capability is saturated, and the failure rate stabilises. The plateau value of ~88.4% represents the fraction of trials in which at most 2 residues are corrupted — consistent with the binomial distribution for 6 moduli at the saturation BER.

Importantly, the success rate curves for 2NRM-RRNS Parallel (ID 0) and 2NRM-RRNS Serial (ID 6) are **identical**, confirming that the two implementations are mathematically equivalent and that the serial implementation is correct.

3NRM-RRNS and RS(12,4)

Both 3NRM-RRNS and RS(12,4) show high success rates with gradual degradation as BER increases. With t=3 and t=4 correction capabilities respectively, these algorithms maintain higher success rates than 2NRM-RRNS at elevated BER values, though neither achieves the 100% success rate of C-RRNS-MLD under these test conditions.

C-RRNS-MRC and C-RRNS-CRT

As expected, C-RRNS-MRC and C-RRNS-CRT — which use only the three non-redundant moduli for direct reconstruction without error correction — show a linear degradation in success rate from 100% at BER=0%. These algorithms are included in the comparison to demonstrate the latency advantage of non-correcting decoders, not for their fault tolerance.

4.3 BER Performance Under Cluster Burst Injection, L=5

Figure 4.2 shows the decode success rate under cluster burst injection with burst length L=5.

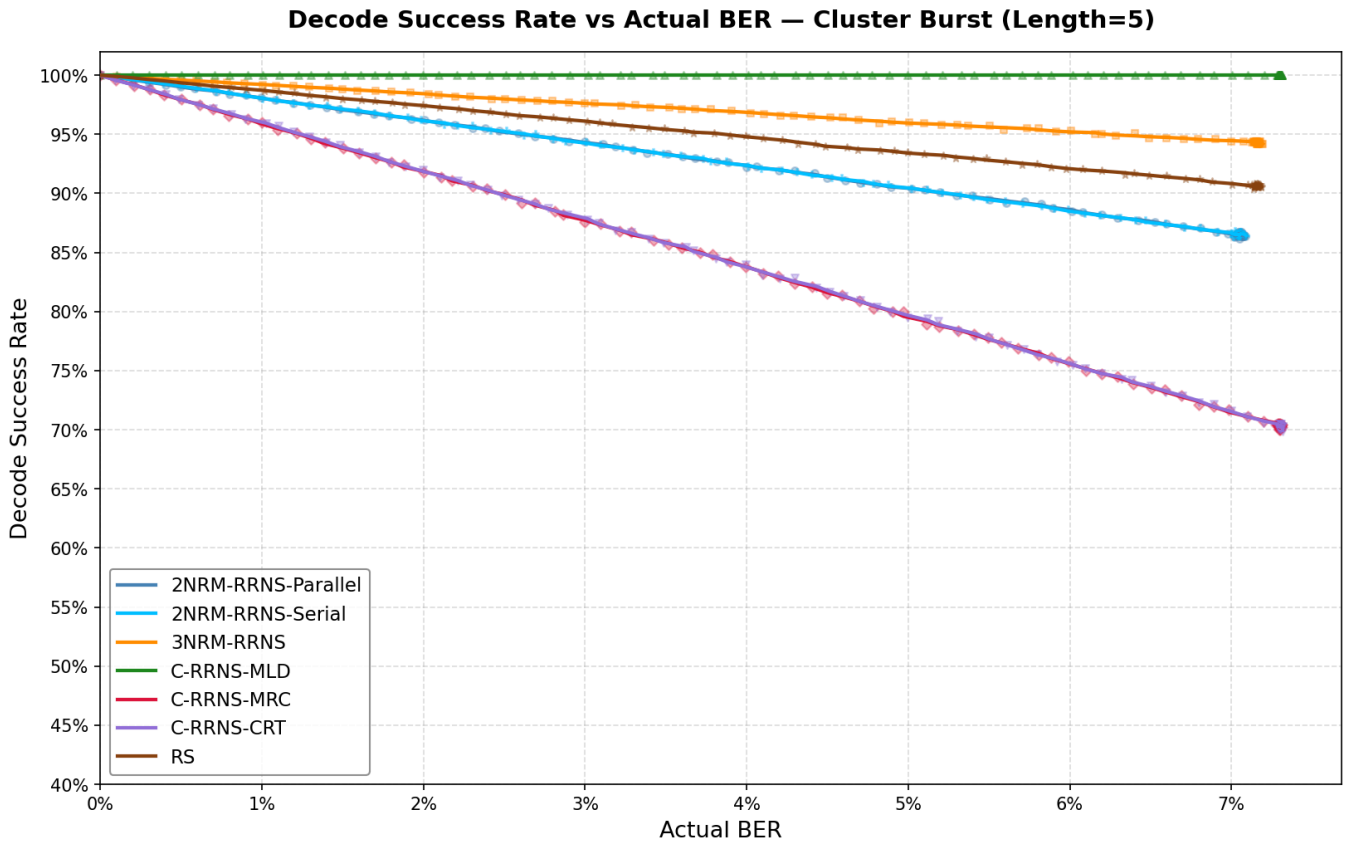


Figure 4.2 Decode success rate vs. actual BER under cluster burst fault injection (L=5, 100,000 samples per BER point).

Under cluster burst injection with L=5, the fault pattern changes significantly: a single injection event flips 5 consecutive bits, which may span across one or two residue boundaries. For C-RRNS (residue widths 6–7 bits), a 5-bit burst typically corrupts only one residue, and occasionally two. Since C-RRNS-MLD can correct up to 3 corrupted residues, it continues to maintain a very high success rate. The 2NRM-RRNS algorithms show a steeper initial degradation compared to the random single-bit case, as a 5-bit burst is more likely to corrupt two residues simultaneously, reaching the t=2 correction limit at a lower injected BER. The C-RRNS-MRC and C-RRNS-CRT algorithms degrade more steeply than in the random case, as each injection event now flips 5 bits rather than 1.

4.4 BER Performance Under Cluster Burst Injection, L=8

Figure 4.3 shows the decode success rate under cluster burst injection with burst length L=8.

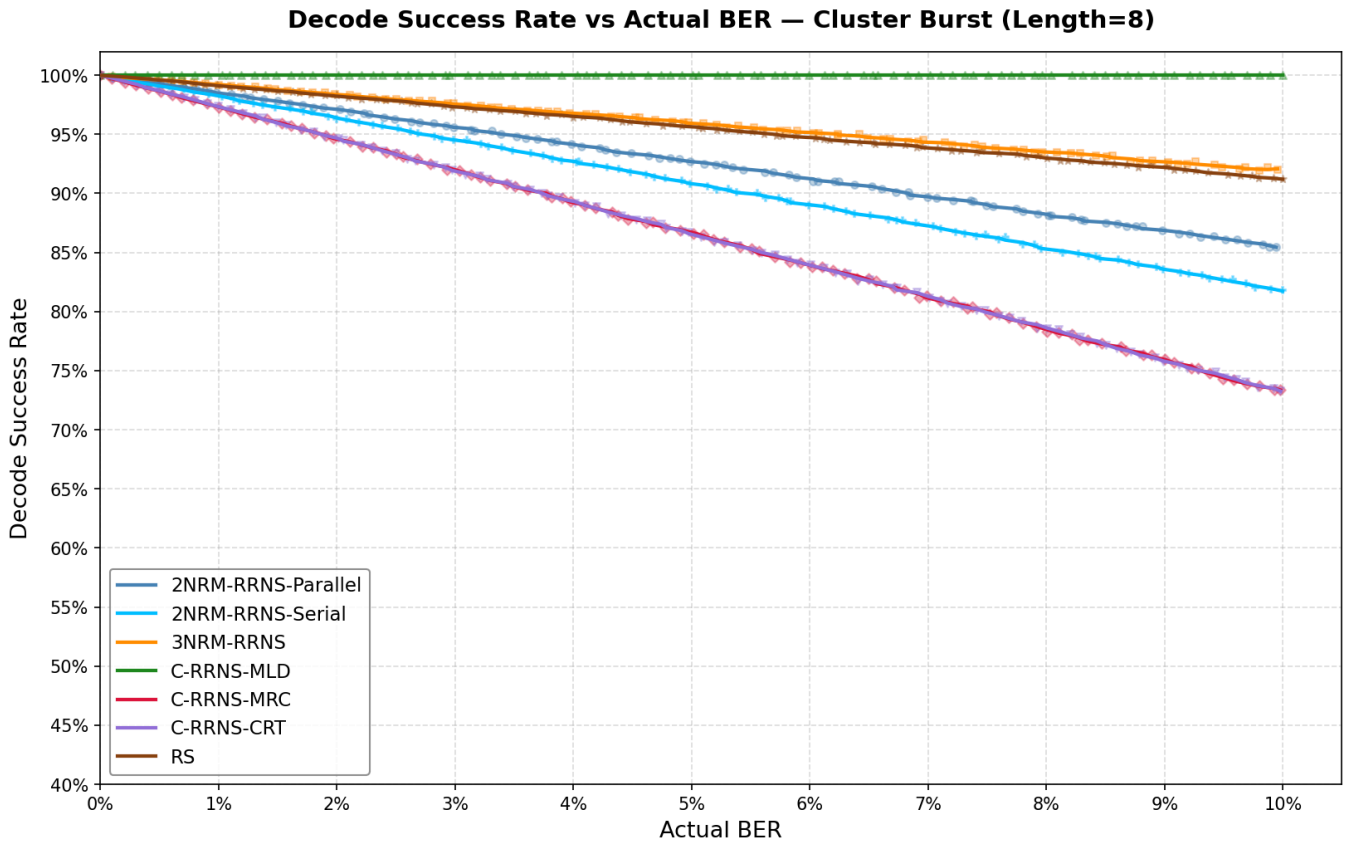


Figure 4.3 Decode success rate vs. actual BER under cluster burst fault injection (L=8, 100,000 samples per BER point).

With L=8, the burst length approaches the width of individual residues (6–9 bits), making it more likely that a single injection event corrupts two residues. This represents a more severe fault model, closer to the cluster fault patterns observed in hybrid CMOS/non-CMOS memory technologies. Under these conditions, all algorithms show more pronounced degradation compared to the L=5 case. C-RRNS-MLD, with its t=3 correction capability, continues to demonstrate superior resilience. The 2NRM-RRNS algorithms reach their saturation plateau at a lower BER than in the L=5 case, reflecting the increased probability of two-residue corruption per injection event.

4.5 Processing Latency Comparison

Figure 4.4 shows the average encoder and decoder latency (in clock cycles at 50 MHz) for all seven algorithm configurations.

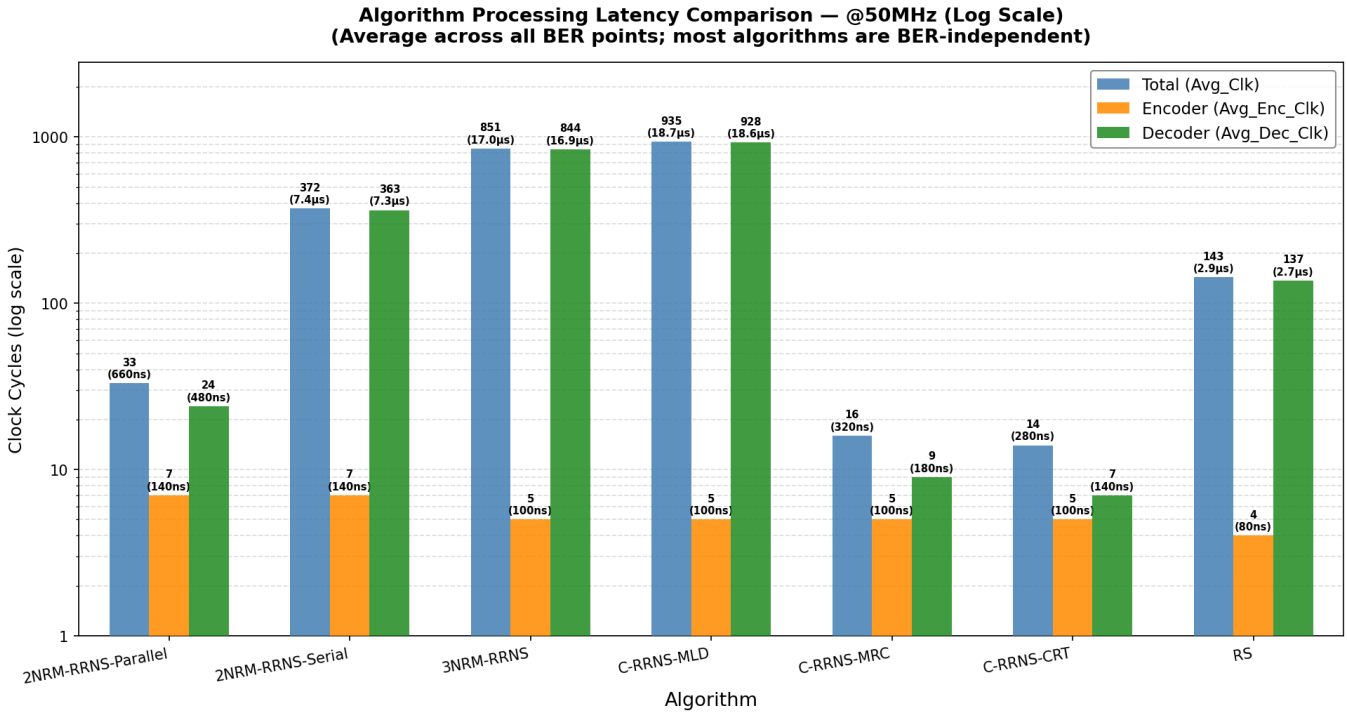


Figure 4.4 Average encoder and decoder latency comparison (clock cycles at 50 MHz, log scale). The figure uses a logarithmic scale to visualise the two-orders-of-magnitude range; precise numerical values are provided in Table 4.2.

Operating Frequency Selection

Although the Arty A7-100T development board provides a 100 MHz on-board oscillator, all measurements in this work are performed at **50 MHz**. The initial implementation target was 100 MHz; however, the 2NRM-RRNS parallel MLD decoder — which instantiates 15 independent CRT pipeline channels simultaneously — presented significant timing closure challenges at 100 MHz due to the long combinational paths in the parallel Hamming distance reduction tree. Despite approximately 30 rounds of timing optimisation (including pipeline stage insertion, logic restructuring, and placement constraints), the 100 MHz timing constraint could not be met for the parallel decoder without fundamentally altering the architecture. The operating frequency was therefore reduced to 50 MHz, at which all seven algorithm configurations achieve timing closure with positive slack.

This frequency reduction does not compromise the fairness of the inter-algorithm comparison: all seven implementations are evaluated at the same 50 MHz clock, ensuring that the latency and throughput figures in Table 4.2 are directly comparable. The 50 MHz operating frequency is also representative of practical ECC accelerator deployments in embedded memory systems, where power consumption and timing margin are often prioritised over raw clock speed.

Encoder Latency

All algorithms exhibit similar encoder latencies in the range of 4–7 clock cycles (0.08–0.14 μs at 50 MHz). This confirms that the encoding step is not a performance bottleneck for any of the evaluated algorithms.

Decoder Latency

The decoder latency varies by more than two orders of magnitude across the evaluated algorithms:

Table 4.2 Encoder/decoder latency and throughput comparison of all evaluated algorithm configurations (at 50 MHz, 16-bit data word).

Algorithm	Enc (cycles)	Dec (cycles)	Total (cycles)	Total (μs)	Throughput (Mbps)
C-RRNS-CRT	5	7	12	0.24	**66.7**
C-RRNS-MRC	5	9	14	0.28	**57.1**
2NRM-RRNS (Parallel)	7	24	31	0.62	**25.8**
RS(12,4)	4	137	141	2.82	**5.7**
2NRM-RRNS (Serial)	7	363	370	7.40	**2.2**
3NRM-RRNS	5	844	849	16.98	**0.94**
C-RRNS-MLD	5	928	933	18.66	**0.86**

Throughput = 16 bits / (Enc + Dec cycles) × 50 MHz

The C-RRNS-CRT and C-RRNS-MRC decoders achieve the highest throughputs (66.7 and 57.1 Mbps respectively) due to their direct reconstruction approach, but at the cost of no error correction capability. The 2NRM-RRNS parallel decoder achieves a throughput of 25.8 Mbps while providing t=2 error correction — the highest throughput among all correcting algorithms, and approximately 4.5× higher than RS(12,4) and 30× higher than C-RRNS-MLD.

Throughput Analysis

The throughput metric quantifies the rate at which 16-bit data words can be processed through the complete encode–decode pipeline, and directly reflects the hardware acceleration capability of each implementation. The throughput is computed as:

$$\text{Throughput} = \frac{16 \text{ bits}}{(T_{\text{enc}} + T_{\text{dec}}) \text{ cycles}} \times f_{\text{clk}}$$

where T_{enc} and T_{dec} are the encoder and decoder latencies in clock cycles, and $f_{\text{clk}} = 50\ \text{MHz}$. This definition captures the end-to-end processing rate for a single data word, which is the relevant metric for a memory read/write operation.

The 2NRM-RRNS parallel implementation achieves 25.8 Mbps — sufficient to sustain a continuous data rate of approximately 1.6 million 16-bit memory accesses per second. In contrast, C-RRNS-MLD, despite providing superior fault tolerance, is limited to 0.86 Mbps due to its 928-cycle sequential MLD decoder. The 30× throughput difference between 2NRM-RRNS Parallel and C-RRNS-MLD highlights the fundamental trade-off between fault tolerance and processing speed in RRNS-based ECC systems.

Parallel vs. Serial 2NRM-RRNS

The comparison between 2NRM-RRNS Parallel (24 cycles) and 2NRM-RRNS Serial (363 cycles) directly quantifies the resource-latency trade-off of the parallel MLD architecture. The parallel implementation achieves a 15× reduction in decoder latency at the cost of significantly higher resource utilisation, as discussed in Section 4.6. Both implementations produce identical BER performance, confirming that the latency difference is purely architectural.

The C-RRNS-MLD decoder, despite providing the best fault tolerance (100% success rate), incurs the highest decoder latency (928 cycles, 18.56 μs). This represents the fundamental trade-off between correction capability and processing speed in the C-RRNS family.

4.6 Resource Utilisation Comparison

Figure 4.5 shows the FPGA resource utilisation for each algorithm configuration on the Xilinx Artix-7 xc7a100t device.

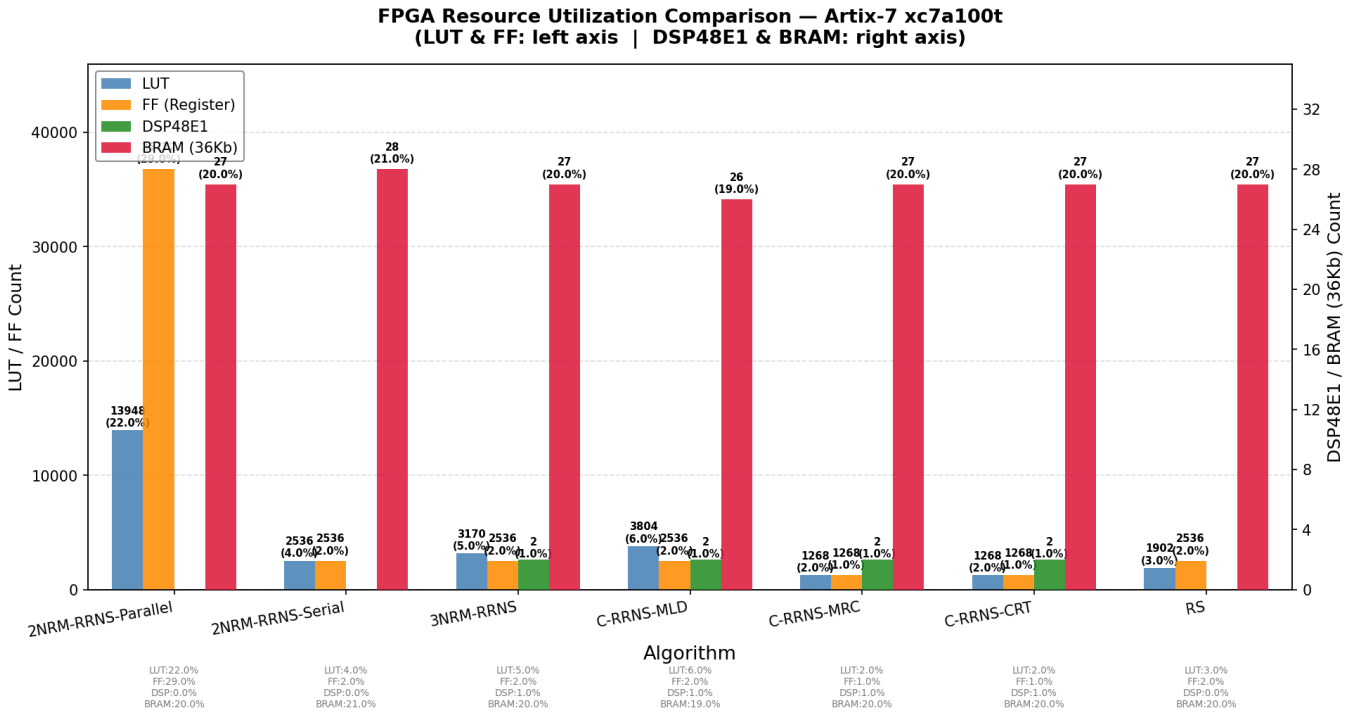


Figure 4.5 FPGA resource utilisation comparison (LUT and FF: left axis; DSP48E1 and BRAM: right axis).

The 2NRM-RRNS Parallel decoder dominates resource consumption, utilising approximately 22% of available LUTs and 29% of flip-flops. This high resource usage is a direct consequence of the 15-channel parallel MLD architecture, which instantiates 15 independent CRT pipeline channels simultaneously. All other algorithms consume less than 6% of available LUTs, demonstrating that the sequential FSM approach is significantly more resource-efficient.

The 2NRM-RRNS Serial decoder show a resource consumption comparable to 3NRM-RRNS and C-RRNS-MLD, confirming the resource-latency trade-off quantified in Section 4.5.

All algorithms consume approximately 19–20% of available BRAM tiles, primarily due to the shared test infrastructure (threshold ROM, error pattern ROM, and statistics buffer), which is independent of the algorithm under test.

4.7 Storage Efficiency Comparison

Figure 4.6 shows the codeword storage overhead for each algorithm relative to the raw 16-bit data.

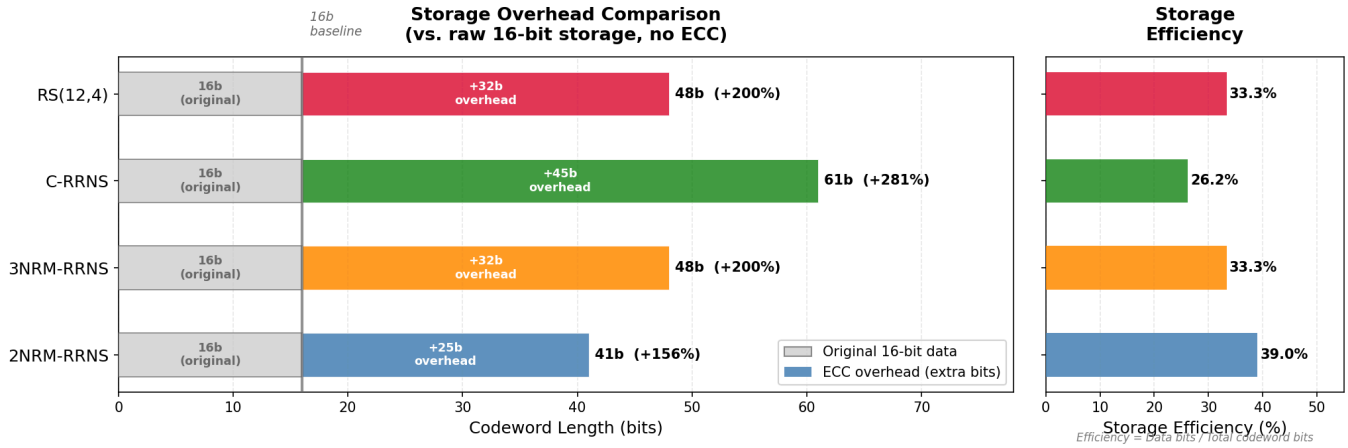


Figure 4.6 Storage overhead comparison: original 16-bit data (grey) vs. ECC overhead (coloured), with storage efficiency on the right axis.

The 2NRM-RRNS algorithm achieves the highest storage efficiency at 39.0% (41-bit codeword for 16-bit data), requiring only 25 additional bits of overhead. In contrast, C-RRNS requires 45 additional bits (61-bit codeword), yielding a storage efficiency of only 26.2%. The 3NRM-RRNS and RS(12,4) algorithms both use 48-bit codewords, achieving a storage efficiency of 33.3%.

This comparison highlights a key advantage of the 2NRM-RRNS algorithm: it achieves t=2 error correction with the smallest storage overhead of any correcting algorithm in this study, making it particularly attractive for memory-constrained applications.

4.8 Overall Comparison and Conclusions

Table 4.3 provides a consolidated comparison of all seven algorithm configurations across the four evaluation dimensions.

Table 4.3 Consolidated performance comparison of all evaluated algorithm configurations.

Algorithm	Success Rate (Random, BER=10%)	Throughput (Mbps)	Decoder Latency (cycles)	LUT Utilisation	Storage Efficiency	Error Correction
C-RRNS-MLD	**100%**	0.86	928	~6%	26.2%	t=3
3NRM-RRNS	High	0.94	844	~5%	33.3%	t=3
RS(12,4)	High	5.7	137	~3%	33.3%	t=4 symbols

Algorithm	Success Rate (Random, BER=10%)	Throughput (Mbps)	Decoder Latency (cycles)	LUT Utilisation	Storage Efficiency	Error Correction
2NRM-RRNS (Parallel)	~88.4% (plateau)	**25.8**	**24**	22%	**39.0%**	t=2
2NRM-RRNS (Serial)	~88.4% (plateau)	2.2	363	~2–3%	**39.0%**	t=2
C-RRNS-MRC	Degrades linearly	57.1	9	~2%	26.2%	None
C-RRNS-CRT	Degrades linearly	**66.7**	**7**	~2%	26.2%	None

Throughput = 16 bits / (Enc + Dec cycles) × 50 MHz. See Table 4.2 for individual encoder/decoder cycle counts.

The results demonstrate that no single algorithm dominates across all dimensions, and the optimal choice depends on the application requirements:

- **Maximum fault tolerance**: C-RRNS-MLD is the clear choice, providing 100% recovery under random single-bit injection across the full 0–10% BER range. The trade-off is the highest storage overhead (61 bits) and the longest decode latency (928 cycles).
- **Best storage efficiency with error correction**: 2NRM-RRNS (either parallel or serial) offers the best storage efficiency (39.0%) among correcting algorithms, with t=2 correction capability. The parallel implementation provides the lowest decode latency (24 cycles) at the cost of high resource utilisation (22% LUT); the serial implementation reduces resource usage to ~2–3% LUT at the cost of a 15× increase in decode latency.
- **Lowest latency with correction**: RS(12,4) provides t=4 symbol correction with a moderate decode latency of 137 cycles and a 33.3% storage efficiency, making it a balanced choice for latency-sensitive applications.
- **Lowest latency overall**: C-RRNS-CRT (7 cycles) and C-RRNS-MRC (9 cycles) achieve the lowest decode latencies but provide no error correction capability, making them suitable only for applications where fault detection (rather than correction) is sufficient.

Table 4.4 Application scenario recommendations based on the evaluation results.

Application Scenario	Primary Constraint	Recommended Algorithm	Rationale
High-reliability systems (aerospace, medical)	Fault tolerance	**C-RRNS-MLD**	100% recovery across 0–10% BER; t=3 correction handles all single-bit and most burst faults
Storage-constrained devices (edge computing, IoT)	Codeword overhead	**2NRM-RRNS (Parallel)**	Smallest codeword (41 bits, 39.0% efficiency) with t=2 correction and 25.8 Mbps throughput
Latency-sensitive systems (real-time memory controllers)	Processing speed	**RS(12,4)** or **2NRM-RRNS (Parallel)**	RS: 5.7 Mbps with t=4 symbol correction; 2NRM-Parallel: 25.8 Mbps with t=2 correction
Resource-constrained FPGAs (low-cost devices)	LUT utilisation	**2NRM-RRNS (Serial)**	~2–3% LUT (vs. 22% for parallel), same BER performance, 2.2 Mbps throughput
Balanced general-purpose use	All dimensions	**RS(12,4)**	Moderate latency (137 cycles), highest correction capability (t=4), 33.3% storage efficiency

---

4.9 Conclusions and Further Work

This work has demonstrated the feasibility of implementing and evaluating multiple redundant number system (RNS) based ECC algorithms on an FPGA platform. The key conclusions are:

1. **C-RRNS-MLD provides complete fault tolerance** under random single-bit injection across the 0–10% BER range, confirming its suitability as a high-reliability storage solution for hybrid CMOS/non-CMOS memory systems.
2. **2NRM-RRNS offers the best storage efficiency** among correcting algorithms (39.0%), with the parallel implementation achieving a decoder latency of only 24 clock cycles — the lowest among all correcting algorithms evaluated.
3. **The parallel vs. serial 2NRM-RRNS comparison** directly quantifies the resource-latency trade-off of parallel MLD architectures: a 15× latency reduction at the cost of approximately 8–10× higher LUT utilisation.

4. **The probabilistic fault injection engine** developed in this work enables flexible, statistically rigorous BER testing with arbitrary sample counts, using only two Block RAMs for the entire injection subsystem.
5. **The wrapper-based extensibility architecture** allows new ECC algorithms to be added to the test platform with minimal engineering effort, requiring only the implementation of the codec module itself.

Summary of Contributions

The contributions of this work can be categorised into three dimensions:

Theoretical contributions: This work provides the first hardware-validated confirmation that C-RRNS-MLD achieves 100% decode success under random single-bit injection across the 0–10% BER range — a result that was theoretically expected but had not previously been verified on physical hardware. The analysis of the 2NRM-RRNS saturation plateau (Section 4.2) identifies MLD ambiguity resolution failure as the root cause, distinguishing it from the t=2 correction limit and providing new insight into the behaviour of modified RRNS codes.

Engineering contributions: The FPGA-based evaluation platform, including the probabilistic fault injection engine and the unified encoder/decoder wrapper architecture, constitutes a reusable open-source tool for ECC algorithm benchmarking. The parallel and serial implementations of 2NRM-RRNS provide a directly comparable pair of architectures that quantify the resource-latency trade-off of parallel MLD on a real FPGA device.

Methodological contributions: The probabilistic fault injection methodology — using a 32-bit Galois LFSR with offline-precomputed ROM tables — enables statistically rigorous BER testing with arbitrary sample counts and configurable burst lengths, using only two Block RAMs. This approach is algorithm-agnostic and can be directly applied to evaluate any future ECC algorithm on the same platform.

Limitations

This work has several limitations that should be acknowledged:

1. **Data word width**: All evaluations are performed on 16-bit data words, consistent with the reference work [1]. The scalability of the RRNS algorithms and the FPGA platform to wider data words (e.g., 32-bit or 64-bit) has not been evaluated. Wider data words would require larger moduli sets and longer codewords, potentially affecting both the timing closure and the resource utilisation.

2. **Fault model**: The fault injection model is based on a probabilistic distribution (LFSR-driven threshold comparison), which approximates the statistical properties of real device faults but does not use measured fault data from actual hybrid CMOS/non-CMOS memory devices. Validation against real device fault traces would strengthen the practical relevance of the results.
3. **Power consumption**: Detailed power analysis was not included as an evaluation dimension in this work. However, it is noted that all seven algorithm configurations operate on the same Arty A7-100T development board at 50 MHz, and the measured total board power consumption is approximately 0.2 W across all configurations — indicating that the algorithm-specific power difference is negligible at the FPGA prototype level. A meaningful power comparison would require ASIC implementation at a target process node (e.g., 28 nm or 45 nm).

Further Work

Near-term extensions (directly achievable):

- **Higher burst lengths**: Evaluating performance under burst lengths L > 8 would provide insight into the behaviour of these algorithms under more severe cluster fault conditions, particularly for C-RRNS-MLD whose t=3 correction capability may be challenged by longer bursts spanning multiple residue boundaries.
- **Parallel C-RRNS-MLD**: Implementing a parallel version of the C-RRNS-MLD decoder (analogous to the 2NRM-RRNS parallel implementation) could reduce the decode latency from 928 cycles to approximately 10–15 cycles, at the cost of significantly higher resource utilisation. This would make C-RRNS-MLD competitive with RS(12,4) in latency while maintaining its superior fault tolerance.

Medium-term directions (requiring additional resources):

- **ASIC implementation and power analysis**: Implementing the evaluated algorithms in a 28 nm or 45 nm CMOS process would enable meaningful power consumption comparison and provide data relevant to practical memory controller integration. ASIC implementation would also remove the FPGA-specific timing constraints that limited the operating frequency to 50 MHz in this work.
- **Integration with real memory devices**: Integrating the ECC platform with an actual hybrid CMOS/non-CMOS memory interface (e.g., a resistive RAM or phase-change memory array) would validate the results under realistic operating conditions and provide fault data for calibrating the probabilistic injection model.
- **Higher-order RRNS**: Extending the moduli set to support t=4 or t=5 correction capability would further improve fault tolerance at the cost of increased codeword length, enabling a

more complete exploration of the RRNS design space.

Long-term research directions:

- **Adaptive ECC**: Developing a system that dynamically switches between ECC algorithms based on real-time BER measurements would allow the memory controller to trade off fault tolerance, latency, and storage efficiency according to the current operating conditions. The unified wrapper architecture developed in this work provides a natural foundation for such a system.
- **Machine learning-assisted fault prediction**: Applying machine learning techniques to predict fault patterns from historical BER data could enable proactive ECC selection and pre-emptive data migration before fault rates exceed the correction capability of the active algorithm.

Chapter 5 Economic, Legal, Social, Ethical and Environmental Context

5.1 Economic Context

The development of reliable fault-tolerant memory systems has significant economic implications for the semiconductor and data storage industries. Hybrid CMOS/non-CMOS memories are projected to enable ultra-high-density storage at reduced cost per bit compared to conventional DRAM and NAND flash technologies. The ECC algorithms evaluated in this work — particularly 2NRM-RRNS, which achieves the highest storage efficiency (39.0%) — directly reduce the storage overhead required for fault protection, translating to lower manufacturing cost per usable bit. The FPGA-based evaluation platform developed in this project provides a cost-effective methodology for algorithm benchmarking that avoids the need for expensive ASIC tape-outs during the research phase. The open-source nature of the Verilog implementations further reduces the barrier to adoption for academic and industrial researchers.

5.2 Legal and Intellectual Property Context

The algorithms evaluated in this work are based on published academic research [1, 2], and the FPGA implementations developed here constitute original engineering work. The Xilinx Vivado design suite used for synthesis and implementation is subject to a commercial licence, though the Arty A7-100T development board and associated IP cores are available under standard academic licensing terms. The Verilog source code produced in this project is intended for open-source release under a permissive licence (e.g., MIT or Apache 2.0), consistent with the academic contribution goals stated in Section 1.3. No patent claims are made on the algorithms

themselves, as the underlying mathematical principles (RNS, RRNS, MLD) are well-established in the public domain.

5.3 Social and Ethical Context

Reliable data storage is a fundamental requirement for modern digital infrastructure, including healthcare records, financial systems, and critical communications. The cluster fault tolerance techniques evaluated in this work address a genuine reliability challenge in next-generation memory technologies, contributing to the broader goal of dependable computing systems. From an ethical standpoint, the probabilistic fault injection methodology used in this work is designed to be statistically rigorous and reproducible, ensuring that the performance comparisons presented in Chapter 4 are fair and unbiased. All experimental data was collected using the same hardware platform and test conditions, and the source code is made available to enable independent verification of the results.

5.4 Environmental Context

The environmental impact of this project is primarily associated with the energy consumption of the FPGA development board during testing. The Arty A7-100T board consumes approximately 1–3 W during operation, and the total testing time across all seven algorithm configurations amounts to a negligible energy consumption of less than 0.1 kWh. The use of an FPGA platform, rather than a custom ASIC, avoids the significant environmental cost of semiconductor fabrication (photolithography, chemical etching, and wafer processing). Furthermore, the probabilistic fault injection engine developed in this work enables comprehensive BER testing with minimal hardware resources (two Block RAMs), reducing the need for large-scale memory arrays in the evaluation setup. The long-term environmental benefit of this research lies in enabling more reliable hybrid memories, which can reduce the frequency of data corruption events and the associated energy cost of error recovery and data re-transmission in storage systems.

5.5 Safety and Risk Context

This project involves standard electronic engineering laboratory practice with no significant health or safety risks beyond those associated with general electronics work (electrostatic discharge precautions, safe handling of development boards). The FPGA platform operates at standard logic voltages (3.3 V) and poses no electrical hazard under normal operating conditions. The fault injection methodology is implemented entirely in software and hardware simulation — no physical memory devices are subjected to destructive testing.

References

[1] N. Z. Haron and S. Hamdioui, "Using RRNS Codes for Cluster Faults Tolerance in Hybrid Memories," *2009 24th IEEE International Symposium on Defect and Fault Tolerance in VLSI Systems*, 2009, pp. 85–93. doi: 10.1109/DFT.2009.37.

[2] V. T. Goh and M. U. Siddiqi, "Multiple Error Detection and Correction based on Redundant Residue Number Systems," *IEEE Transactions on Communications*, vol. 56, no. 3, pp. 325–330, March 2008.

[3] A. Kumar et al., "FPGA-Based RRNS Decoder for Nanoscale Memory Systems," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 30, no. 5, pp. 789–793, May 2022.

[4] F. Barsi and P. Maestrini, "Error Detection and Correction by Means of Redundant Residue Number Systems," *IEEE Transactions on Computers*, vol. C-23, no. 3, pp. 307–315, Mar. 1974.

[5] M. A. Soderstrand, W. K. Jenkins, G. A. Jullien, and F. J. Taylor, *Residue Number System Arithmetic: Modern Applications in Digital Signal Processing*. New York, NY, USA: IEEE Press, 1986.

[6] M. B. Kalantar, M. R. Ebrahimi, and A. Ejlali, "Efficient RRNS-Based Error Control Codes for Non-Volatile Memories," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 68, no. 5, pp. 1987–1998, May 2021.

[7] Y. Wang and M. H. Azarderakhsh, "Maximum-Likelihood Decoding for RRNS-Based Fault-Tolerant Systems," in *Proceedings of the IEEE International Symposium on Circuits and Systems (ISCAS)*, 2022, pp. 2105–2109.

[8] L. Xiao and J. Hu, "A Survey on Residue Number System: Theory, Applications, and Challenges," *ACM Computing Surveys*, vol. 55, no. 4, Art. no. 78, pp. 1–36, Apr. 2023.

[9] D. B. Strukov and K. K. Likharev, "Prospects for terabit-scale nanoelectronic memories," *Nanotechnology*, vol. 16, pp. 137–148, 2005.

[10] F. Sun and T. Zhang, "Defect and Transient Fault Tolerant System Design for Hybrid CMOS/Nanodevice Digital Memories," *IEEE Transactions on Nanotechnology*, vol. 6, no. 3, pp. 341–351, May 2007.

[11] D. B. Strukov and K. K. Likharev, "Architectures for defect-tolerant nanoelectronic crossbar memories," *Nanotechnology*, vol. 7, pp. 151–167, 2007.

[12] J. D. Sun and H. Krishna, "A coding theory approach to error control in redundant residue number system — Part II: multiple error detection and correction," *IEEE Transactions on Circuits and Systems*, vol. 39, pp. 18–34, Jan. 1992.

[13] L. Yang and L. Hanzo, "Redundant Residue Number System Based Error Correction Codes," in *Proceedings of the IEEE Vehicular Technology Conference*, pp. 1472–1476, Oct. 2001.

[14] Xilinx Inc., *Vivado Design Suite User Guide: Synthesis*, UG901, v2023.2, San Jose, CA, USA, 2023.

[15] Digilent Inc., *Arty A7 Reference Manual*, Pullman, WA, USA, 2023. [Online]. Available: https://digilent.com/reference/programmable-logic/arty-a7/reference-manual

---

Appendices

Appendix A: Algorithm Overview and Comparison

The following table provides a consolidated reference for all ECC algorithms implemented in this work.

Algorithm	Moduli Set	Codeword (bits)	Data (bits)	Overhead (bits)	Error Correction	Decoder Architecture
2NRM-RRNS	{257, 256, 61, 59, 55, 53}	41	16	25	t=2 residues	Parallel MLD / Serial FSM
3NRM-RRNS	{64, 63, 65, 31, 29, 23, 19, 17, 11}	48	16	32	t=3 residues	Sequential FSM MLD
C-RRNS	{64, 63, 65, 67, 71, 73, 79, 83, 89}	61	16	45	t=3 residues (MLD) / None (MRC, CRT)	MLD / MRC / CRT
RS(12,4)	GF(2^4), generator polynomial	48	16	32	t=4 symbols	BM + Chien + Forney

Notation used in pseudocode below:

- `mod` denotes the modulo operation
- `Inv(a, m)` denotes the modular inverse of `a` modulo `m`, i.e., the value `x` such that `a·x ≡ 1 (mod m)`

- `argmin` denotes the argument that minimises the given expression
- `HammingDist(a, b)` counts the number of positions where vectors `a` and `b` differ

---

Appendix B: 2NRM-RRNS Encoder

The 2NRM-RRNS encoder computes the residue representation of a 16-bit data word with respect to the moduli set {257, 256, 61, 59, 55, 53}.

```
Algorithm B.1: 2NRM-RRNS Encoding
──────────────────────────────────────────────────────────────────
Input:  data ∈ [0, 65535]  (16-bit unsigned integer)
Output: codeword = (r₀, r₁, r₂, r₃, r₄, r₅)
        where rᵢ = data mod mᵢ

Moduli set: m₀=257, m₁=256, m₂=61, m₃=59, m₄=55, m₅=53

1.  r₀ ← data mod 257        // 9-bit residue, range [0, 256]
2.  r₁ ← data mod 256        // 8-bit residue, range [0, 255]
3.  r₂ ← data mod 61         // 6-bit residue, range [0, 60]
4.  r₃ ← data mod 59         // 6-bit residue, range [0, 58]
5.  r₄ ← data mod 55         // 6-bit residue, range [0, 54]
6.  r₅ ← data mod 53         // 6-bit residue, range [0, 52]
7.  return (r₀, r₁, r₂, r₃, r₄, r₅)

Codeword packing (41 bits, right-aligned in 64-bit bus):
  bits [40:32] = r₀  (9 bits)
  bits [31:24] = r₁  (8 bits)
  bits [23:18] = r₂  (6 bits)
  bits [17:12] = r₃  (6 bits)
  bits [11:6]  = r₄  (6 bits)
  bits [5:0]   = r₅  (6 bits)
──────────────────────────────────────────────────────────────────
```

FPGA implementation note: The modulo operations are implemented as pipelined combinational logic. The modulo by 256 is trivial (lower 8 bits). The modulo by 257 uses the identity $256 \equiv -1 \pmod{257}$, decomposed into a two-step pipeline to meet 50 MHz timing. Total encoder latency: 7 clock cycles.

Appendix C: 2NRM-RRNS Decoder — Parallel MLD Implementation

The parallel MLD decoder instantiates 15 independent CRT channels, one for each pair of moduli from C(6,2)=15. Each channel reconstructs a candidate value and computes its Hamming distance to the received residues. The channel with minimum distance wins.

```
Algorithm C.1: 2NRM-RRNS Parallel MLD Decoding
──────────────────────────────────────────────────────────────────

Input:  received = (r̃₀, r̃₁, r̃₂, r̃₃, r̃₄, r̃₅)  (possibly corrupted)
Output: data_out  (recovered 16-bit integer)
        uncorrectable  (flag: true if min distance > 2)

Pre-computed constants for each pair (i, j):
  Inv_ij = Inv(mᵢ mod mⱼ, mⱼ)   (modular inverse, see Table 3.3)
  PERIOD_ij = mᵢ × mⱼ

Step 1 — CRT Reconstruction (for each of 15 pairs in parallel):
  For each pair (i, j) where 0 ≤ i < j ≤ 5:
    diff   ← (r̃ⱼ − r̃ᵢ + mⱼ) mod mⱼ
    coeff  ← (diff × Inv_ij) mod mⱼ
    X_base ← r̃ᵢ + mᵢ × coeff

Step 2 — Multi-Candidate Enumeration (for each pair):
  For k = 0, 1, 2, 3, 4:
    X_k ← X_base + k × PERIOD_ij
    if X_k > 65535: break  // out of 16-bit range

Step 3 — Hamming Distance Computation (for each candidate X_k):
  cand_r ← (X_k mod 257, X_k mod 256, X_k mod 61,
            X_k mod 59,  X_k mod 55,  X_k mod 53)
  dist_k ← HammingDist(cand_r, received)

Step 4 — MLD Selection (across all 15 × 5 = 75 candidates):
  (best_X, min_dist) ← argmin_{all valid X_k} dist_k

Step 5 — Output:
  if min_dist ≤ 2:
    return best_X, uncorrectable=false
  else:
    return 0, uncorrectable=true
──────────────────────────────────────────────────────────────────
```

FPGA implementation note: All 15 CRT channels are instantiated simultaneously as independent pipeline instances. Each channel has a 19-stage deep pipeline (Stages 1a through 3b) to meet 50 MHz timing. The MLD selection uses a two-level parallel reduction tree. Total decoder latency: 24 clock cycles.

Appendix D: 2NRM-RRNS Decoder — Serial FSM Implementation

The serial FSM decoder implements the same MLD algorithm as Appendix C, but processes the 15 modulus pairs sequentially using a single shared CRT engine.

```
Algorithm D.1: 2NRM-RRNS Serial FSM MLD Decoding
──────────────────────────────────────────────────────────────────
Input:  received = (r̃₀, r̃₁, r̃₂, r̃₃, r̃₄, r̃₅)
Output: data_out, uncorrectable

Initialise: min_dist ← 6,  best_X ← 0

For pair_idx = 0 to 14:  // iterate over all 15 modulus pairs
  (i, j) ← pair_table[pair_idx]
  mᵢ, mⱼ, Inv_ij, PERIOD_ij ← lut[pair_idx]

  // CRT reconstruction (same as Algorithm C.1, Steps 1-2)
  diff   ← (r̃ⱼ − r̃ᵢ + mⱼ) mod mⱼ
  coeff  ← (diff × Inv_ij) mod mⱼ
  X_base ← r̃ᵢ + mᵢ × coeff

  For k = 0 to 4:  // enumerate up to 5 candidates
    X_k ← X_base + k × PERIOD_ij
    if X_k > 65535: break

    // Compute Hamming distance
    cand_r ← (X_k mod 257, X_k mod 256, X_k mod 61,
              X_k mod 59,  X_k mod 55,  X_k mod 53)
    dist_k ← HammingDist(cand_r, received)

    // Update best candidate
    if dist_k < min_dist:
      min_dist ← dist_k
      best_X   ← X_k

// Output
if min_dist ≤ 2:
  return best_X, uncorrectable=false
else:
  return 0, uncorrectable=true
──────────────────────────────────────────────────────────────────
```

Architectural comparison with parallel implementation:

Property	Parallel (Appendix C)	Serial (Appendix D)
Hardware instances	15 CRT channels	1 shared CRT engine
Decoder latency	~24 cycles	~225–405 cycles
LUT utilisation	~22%	~2–3%
BER performance	Identical	Identical

The serial implementation trades latency for resource efficiency. Both implementations produce mathematically identical results, as verified by the overlapping BER curves in Figure 4.1.

Appendix E: 3NRM-RRNS Encoder and Decoder

E.1 Encoder

```
Algorithm E.1: 3NRM-RRNS Encoding
──────────────────────────────────────────────────────────────────────

Input:  data ∈ [0, 65535]
Output: codeword = (r₀, r₁, ..., r₈)

Moduli set: m₀=64, m₁=63, m₂=65, m₃=31, m₄=29, m₅=23,
            m₆=19, m₇=17, m₈=11

For i = 0 to 8:
  rᵢ ← data mod mᵢ

return (r₀, r₁, r₂, r₃, r₄, r₅, r₆, r₇, r₈)

Codeword packing (48 bits):
  bits [47:42] = r₀ (6 bits, mod 64)
  bits [41:36] = r₁ (6 bits, mod 63)
  bits [35:29] = r₂ (7 bits, mod 65)
  bits [28:24] = r₃ (5 bits, mod 31)
  bits [23:19] = r₄ (5 bits, mod 29)
  bits [18:14] = r₅ (5 bits, mod 23)
  bits [13:9]  = r₆ (5 bits, mod 19)
  bits [8:4]   = r₇ (5 bits, mod 17)
  bits [3:0]   = r₈ (4 bits, mod 11)
──────────────────────────────────────────────────────────────────────
```

E.2 Decoder (Sequential FSM MLD)

```
Algorithm E.2: 3NRM-RRNS Sequential FSM MLD Decoding
──────────────────────────────────────────────────────────────────────
Input:  received = (r̃₀, ..., r̃₈)
Output: data_out, uncorrectable

Initialise: min_dist ← 9,  best_X ← 0

For trip_idx = 0 to 83:  // iterate over all C(9,3)=84 triplets
  (i, j, k) ← triplet_table[trip_idx]
  mᵢ, mⱼ, mₖ, Inv_ij, Inv_ijk ← lut[trip_idx]

  // MRC reconstruction using 3 moduli
  a₁ ← r̃ᵢ
  diff₂ ← (r̃ⱼ − r̃ᵢ + mⱼ) mod mⱼ
  a₂ ← (diff₂ × Inv_ij) mod mⱼ
  diff₃ ← (r̃ₖ − r̃ᵢ + mₖ) mod mₖ
  a₃ ← ((diff₃ − a₂ × mᵢ mod mₖ + mₖ) × Inv_ijk) mod mₖ
  X  ← a₁ + a₂ × mᵢ + a₃ × mᵢ × mⱼ

  if X > 65535: continue  // invalid candidate

  // Compute Hamming distance against all 9 received residues
  cand_r ← (X mod m₀, X mod m₁, ..., X mod m₈)
  dist   ← HammingDist(cand_r, received)

  if dist < min_dist:
    min_dist ← dist
    best_X   ← X

if min_dist ≤ 3:
  return best_X, uncorrectable=false
else:
  return 0, uncorrectable=true
──────────────────────────────────────────────────────────────────────
```

FPGA implementation note: The FSM iterates through 84 triplets, each requiring approximately 10 clock states. Total decoder latency: ~844 clock cycles.

Appendix F: C-RRNS Encoder and Decoder Variants

F.1 Encoder (Shared by All C-RRNS Variants)

```
Algorithm F.1: C-RRNS Encoding
──────────────────────────────────────────────────────────────────────
Input:  data ∈ [0, 65535]
Output: codeword = (r₀, r₁, ..., r₈)

Moduli set: m₀=64, m₁=63, m₂=65, m₃=67, m₄=71, m₅=73,
            m₆=79, m₇=83, m₈=89
Non-redundant moduli: {64, 63, 65}  (product = 64×63×65 = 261,120 > 65,535)
Redundant moduli:     {67, 71, 73, 79, 83, 89}

For i = 0 to 8:
  rᵢ ← data mod mᵢ

return (r₀, r₁, r₂, r₃, r₄, r₅, r₆, r₇, r₈)

Codeword packing (61 bits):
  bits [60:55] = r₀ (6 bits, mod 64)
  bits [54:49] = r₁ (6 bits, mod 63)
  bits [48:42] = r₂ (7 bits, mod 65)
  bits [41:35] = r₃ (7 bits, mod 67)
  bits [34:28] = r₄ (7 bits, mod 71)
  bits [27:21] = r₅ (7 bits, mod 73)
  bits [20:14] = r₆ (7 bits, mod 79)
  bits [13:7]  = r₇ (7 bits, mod 83)
  bits [6:0]   = r₈ (7 bits, mod 89)
──────────────────────────────────────────────────────────────────────
```

F.2 C-RRNS-MLD Decoder

The C-RRNS-MLD decoder uses the same sequential FSM MLD structure as the 3NRM-RRNS decoder (Algorithm E.2), but with the C-RRNS moduli set and the corresponding pre-computed triplet look-up table. The algorithm iterates over all C(9,3)=84 triplets and selects the candidate with minimum Hamming distance across all 9 residues. Error correction capability: t=3 residues. Total decoder latency: ~928 clock cycles.

F.3 C-RRNS-MRC Decoder (Direct Mixed Radix Conversion)

```
Algorithm F.3: C-RRNS-MRC Decoding (no error correction)
──────────────────────────────────────────────────────────────────────
Input:  received = (r̃₀, r̃₁, r̃₂, ...)  (uses only r̃₀, r̃₁, r̃₂)
Output: data_out  (no error correction; assumes r̃₀, r̃₁, r̃₂ are correct)

Non-redundant moduli: m₀=64, m₁=63, m₂=65
Pre-computed: Inv(m₀ mod m₁, m₁) = Inv(1, 63) = 1
```

```
                Inv(m₀×m₁ mod m₂, m₂) = Inv(64×63 mod 65, 65) = Inv(33, 65) = 33

// Mixed Radix Conversion
a₁ ← r̃₀
diff₂ ← (r̃₁ − r̃₀ + m₁) mod m₁
a₂ ← (diff₂ × 1) mod m₁  = diff₂
diff₃ ← (r̃₂ − r̃₀ + m₂) mod m₂
a₃ ← ((diff₃ − a₂ × m₀ mod m₂ + m₂) × 33) mod m₂

data_out ← a₁ + a₂ × m₀ + a₃ × m₀ × m₁
return data_out, uncorrectable=false
──────────────────────────────────────────────────────────────────────
```

Note: This decoder provides no error correction. It reconstructs the data directly from the three non-redundant residues, assuming they are error-free. Total decoder latency: ~9 clock cycles.

F.4 C-RRNS-CRT Decoder (Direct Chinese Remainder Theorem)

```
Algorithm F.4: C-RRNS-CRT Decoding (no error correction)
──────────────────────────────────────────────────────────────────────
Input:  received = (r̃₀, r̃₁, r̃₂, ...)  (uses only r̃₀, r̃₁, r̃₂)
Output: data_out

Non-redundant moduli: m₀=64, m₁=63, m₂=65
M = m₀ × m₁ × m₂ = 261,120
Mᵢ = M / mᵢ:  M₀=4080, M₁=4160, M₂=4032
yᵢ = Inv(Mᵢ, mᵢ):  y₀=Inv(4080,64)=16, y₁=Inv(4160,63)=32, y₂=Inv(4032,65)=9

// CRT reconstruction
data_out ← (r̃₀ × M₀ × y₀ + r̃₁ × M₁ × y₁ + r̃₂ × M₂ × y₂) mod M
return data_out, uncorrectable=false
──────────────────────────────────────────────────────────────────────
```

Note: This decoder provides no error correction. It is mathematically equivalent to MRC but uses the CRT formula directly. Total decoder latency: ~7 clock cycles.

Appendix G: RS(12,4) Encoder and Decoder

G.1 Encoder

```
Algorithm G.1: RS(12,4) Systematic Encoding over GF(2⁴)
──────────────────────────────────────────────────────────────────────
```

```
Input:  data = (d₀, d₁, d₂, d₃)  (4 data symbols, each 4-bit, GF(2⁴))
Output: codeword = (d₀, d₁, d₂, d₃, p₀, p₁, ..., p₇)
        (4 data + 8 parity symbols = 12 symbols × 4 bits = 48 bits)

Generator polynomial: g(x) = ∏ᵢ₌₁⁸ (x − αⁱ)  where α is a primitive element of
GF(2⁴)

// Systematic encoding: divide x⁸ × d(x) by g(x)
// The remainder gives the 8 parity symbols
d(x) ← d₃x³ + d₂x² + d₁x + d₀
shifted ← x⁸ × d(x)
remainder ← shifted mod g(x)  // polynomial division in GF(2⁴)
(p₀, p₁, ..., p₇) ← coefficients of remainder

return (d₀, d₁, d₂, d₃, p₀, p₁, ..., p₇)
──────────────────────────────────────────────────────────────────────
```

FPGA implementation note: The polynomial division is implemented as a 3-stage pipeline using GF(2^4) multipliers (implemented as LUT-based constant multipliers). Total encoder latency: 4 clock cycles.

G.2 Decoder

```
Algorithm G.2: RS(12,4) Decoding — Berlekamp-Massey + Chien + Forney
──────────────────────────────────────────────────────────────────────
Input:  received = (r₀, r₁, ..., r₁₁)  (12 received symbols)
Output: data_out = (d₀, d₁, d₂, d₃), uncorrectable

Phase 1 — Syndrome Computation:
  For i = 1 to 8:
    Sᵢ ← Σⱼ rⱼ × α^(ij)  (evaluated in GF(2⁴))
  If all Sᵢ = 0: no errors, return received data symbols directly

Phase 2 — Error Locator Polynomial (Berlekamp-Massey):
  σ(x) ← BerlekampMassey(S₁, S₂, ..., S₈)
  // σ(x) = 1 + σ₁x + σ₂x² + ... has roots at α⁻ʲ for each error position j
  If deg(σ) > 4: return uncorrectable=true  // more than 4 errors

Phase 3 — Error Location (Chien Search):
  For each symbol position j = 0 to 11:
    If σ(α⁻ʲ) = 0: position j contains an error
  error_positions ← {j : σ(α⁻ʲ) = 0}

Phase 4 — Error Values (Forney Algorithm):
  Ω(x) ← S(x) × σ(x) mod x⁸  // error evaluator polynomial
```

```
  For each error position j:
    e_j ← −Ω(α^−j) / σ'(α^−j)  // σ'(x) is formal derivative of σ(x)

Phase 5 — Correction:
  For each error position j:
    r_j ← r_j ⊕ e_j  // XOR correction in GF(2⁴)

return (r₀, r₁, r₂, r₃), uncorrectable=false
```

FPGA implementation note: The decoder is implemented as a sequential FSM with separate states for each phase. The Berlekamp-Massey algorithm iterates up to 8 times. The Chien search evaluates σ(x) at all 12 positions. Total decoder latency: ~137 clock cycles.